

Universidad Europea Miguel de Cervantes

Escuela Politécnica Superior

Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Detección de Phishing en Correos Electrónicos

Autor: Alejandro Villarrubia García

Tutor: Carmelo González García

2026

Resumen

El phishing se ha consolidado como una de las ciberamenazas más persistentes y dañinas del panorama actual de la ciberseguridad. Este Trabajo Fin de Grado aborda el diseño, la implementación y la evaluación de una aplicación web autocontenida para detectar phishing en correos electrónicos. La solución combina un sistema heurístico explicable con un clasificador neuronal basado en TF-IDF y perceptrón multicapa (MLP). Desarrollada en Python con Streamlit, permite analizar texto pegado, archivos .eml y mensajes importados mediante la API de Gmail, además de entrenar y evaluar modelos en español e inglés. Todo el análisis se ejecuta dentro de la propia aplicación mediante módulos reutilizables, sin depender de un servicio HTTP separado. La memoria estudia el fenómeno del phishing desde su base psicológica y su evolución reciente, revisa el estado del arte en machine learning y deep learning, y presenta los resultados obtenidos durante la validación del prototipo. Los resultados confirman que combinar reglas explicables y aprendizaje supervisado constituye una base funcional y extensible para la detección de este tipo de amenazas.

Palabras clave: phishing, ciberseguridad, ingeniería social, machine learning, deep learning, detección de correo electrónico.

Abstract

Phishing has become one of the most persistent and damaging threats in today's cybersecurity landscape. This Bachelor's Thesis presents the design, implementation and evaluation of a self-contained web application for detecting phishing emails. The solution combines an explainable heuristic system with a neural classifier based on TF-IDF and a multilayer perceptron (MLP). Built in Python with Streamlit, it can analyse pasted text, .eml files and messages imported through the Gmail API, as well as train and evaluate Spanish and English models. All analysis runs inside the application through reusable modules, without relying on a separate HTTP service. The document examines phishing from its psychological foundations and recent evolution, reviews the state of the art in machine learning and deep learning, and reports the results obtained during prototype validation. The results show that combining explainable rules with supervised learning provides a functional and extensible basis for detecting this type of threat.

Keywords: phishing, cybersecurity, social engineering, machine learning, deep learning, email detection.

Agradecimientos

Quiero expresar mi agradecimiento a mi tutor, Carmelo González García, por su orientación, sus comentarios y su apoyo constante durante la realización de este trabajo. Agradezco también a mi familia y amigos su comprensión y ánimo a lo largo de toda la carrera, así como a todas las personas que de una u otra forma han contribuido a que este proyecto fuese posible.

Índice de figuras y tablas

Figuras

- Figura 2.1: Diagrama de flujo swimlane de las fases de un ataque de phishing.
- Figura 2.2: Distribución de vectores iniciales en brechas de datos (2025).
- Figura 2.3: Línea temporal de la campaña CorruptQR.
- Figura 2.4: Evolución de los vectores de phishing más utilizados (2019-2025).
- Figura 2.5: Diagrama circular de técnicas de bypass de MFA en ataques BEC (2025).

Tablas

- Tabla 2.1: Métricas clave por fase (2025).
- Tabla 2.2: Comparativa de modelos.
- Tabla 2.3: Controles por fase.
- Tabla 2.4: Aplicación de los principios de Cialdini en campañas de phishing.
- Tabla 2.5: Eficiencia relativa de los principios de Cialdini en campañas de phishing.
- Tabla 3.1: Comparativa de técnicas clásicas de machine learning.
- Tabla 3.2: Comparativa de técnicas de deep learning y transformers.
- Tabla 4.1: Planificación temporal del proyecto.
- Tabla 4.2: Herramientas y tecnologías utilizadas.

Abreviaturas

API: Conjunto de operaciones que permite integrar la aplicación con servicios externos, como Gmail o Telegram.

AiTM: Adversary-in-the-Middle; ataque que se sitúa entre el usuario y el servicio legítimo para interceptar la autenticación.

BEC: Business Email Compromise; fraude que suplanta a directivos o proveedores.

BERT: Bidirectional Encoder Representations from Transformers; modelo de lenguaje pre-entrenado.

CNN: Red neuronal convolucional (Convolutional Neural Network).

CSV: Formato de datos separados por comas (Comma-Separated Values).

DBIR: Data Breach Investigations Report de Verizon.

DKIM: DomainKeys Identified Mail; firma criptográfica de los correos del dominio remitente.

DL: Deep Learning (aprendizaje profundo).

DMARC: Domain-based Message Authentication, Reporting and Conformance.

EML: Formato de archivo de correo electrónico estándar.

F1: Media armónica entre precisión y recall; métrica de evaluación de clasificadores.

GAN: Red generativa antagónica (Generative Adversarial Network).

GenAI: Inteligencia artificial generativa.

GPT: Generative Pre-trained Transformer.

GRU: Gated Recurrent Unit; tipo de red neuronal recurrente.

IA: Inteligencia artificial.

IC3: Internet Crime Complaint Center del FBI.

INCIBE: Instituto Nacional de Ciberseguridad de España.

LLM: Gran modelo de lenguaje (Large Language Model).

LSTM: Long Short-Term Memory; tipo de red neuronal recurrente.

M365: Microsoft 365; suite de productividad de Microsoft.

MFA: Autenticación multifactor (Multi-Factor Authentication).

ML: Machine Learning (aprendizaje automático).

MLP: Perceptrón multicapa (Multi-Layer Perceptron).

OAuth: Protocolo estándar de autorización delegada.

OSINT: Inteligencia de fuentes abiertas (Open Source Intelligence).

PhaaS: Phishing-as-a-Service; plataformas que ofrecen kits de phishing por suscripción.

QR: Código de respuesta rápida (Quick Response).

RCNN: Red neuronal convolucional recurrente (Recurrent CNN).

RF: Random Forest; algoritmo de aprendizaje por conjuntos de árboles de decisión.

SIEM: Security Information and Event Management; plataforma de correlación de eventos de seguridad.

SMS: Servicio de mensajes cortos (Short Message Service).

SPF: Sender Policy Framework; mecanismo de autenticación del remitente.

SVM: Máquina de vectores de soporte (Support Vector Machine).

TF-IDF: Term Frequency-Inverse Document Frequency; técnica de vectorización de texto.

UEBA: User and Entity Behavior Analytics; análisis del comportamiento de usuarios y entidades.

URL: Localizador uniforme de recursos (Uniform Resource Locator).

WHOIS: Protocolo de consulta de datos de registro de dominios.

XAI: Inteligencia artificial explicable (Explainable AI).

INDICE

Resumen	2
Abstract.....	3
Agradecimientos.....	4
Índice de figuras y tablas.....	5
Abreviaturas.....	6
Introducción	10
Marco teórico del phishing	11
Concepto y contexto actual	11
Fases de un ataque de phishing	11
Preparación y Recopilación de Información (Reconocimiento).....	12
Creación y Lanzamiento del Cebo (Setup y Ataque)	13
Interacción y Explotación (Colección Inicial)	14
Adquisición y Monetización de los Datos	14
Comparación crítica de modelos de fases.....	15
Caso real: campaña CorruptQR (2025).....	15
Puntos de intervención y controles	17
Tipos, clasificación y evolución de los ataques de phishing	17
Introducción a la clasificación	17
Clasificación según el vector de entrega	17
Clasificación según el grado de personalización y objetivo	18
Clasificación según la técnica de explotación	19
Evolución histórica resumida	20
Impacto actual (2025)	20
La ingeniería social como fundamento psicológico y sociotécnico del phishing	20
Principios psicológicos fundamentales explotados	21
Sesgos cognitivos específicos explotados	21
El impacto de la inteligencia artificial generativa en la ingeniería social (2023-2025).	22
Consecuencias organizativas y humanas	23
Implicaciones para la detección.....	23
Estado del arte en técnicas de detección y prevención de phishing basadas en Machine Learning y Deep Learning (2019-2025)	25
Evolución de los enfoques de detección.....	25
Técnicas clásicas de Machine Learning (representativas 2019-2022).....	26
Técnicas de Deep Learning y Transformers (2023-2025)	26
Enfoques híbridos y basados en comportamiento (UEBA).....	28
Limitaciones actuales de los enfoques.....	28

Conclusiones del estado del arte y hueco de investigación	29
Metodología y planificación	29
Enfoque metodológico	29
Planificación temporal del proyecto.....	29
Herramientas y tecnologías.....	30
Validación y control de calidad	31
Aspectos éticos y de protección de datos	31
Diseño e implementación de la aplicación web	32
Alcance del sistema.....	32
Arquitectura general	32
Flujo de análisis	33
Sistema heurístico	33
Clasificador neuronal	34
Interfaz de usuario	34
Diseño modular.....	34
Limitaciones	35
Pruebas, resultados y discusión	36
Pruebas unitarias y de integración	36
Dataset de ejemplo.....	36
Resultados del clasificador neuronal.....	36
Comportamiento del sistema heurístico	36
Prueba funcional de la aplicación web	36
Discusión	37
Conclusiones	37
Trabajo futuro	38
Glosario	39
Referencias.....	40
Anexos	41
Anexo A: Estructura del repositorio.....	41
Anexo B: Fragmentos de código fuente	42
B.1. Fachada del análisis heurístico (heurísticas.py).....	42
B.2. Servicio común de análisis (analysis_service.py)	42
B.3. Núcleo del clasificador neuronal (modelo_neural.py).....	43
Anexo C: Guía resumida de instalación y uso	44
C.1. Uso del prototipo	44
Anexo D: Lista de verificación de la validación	45

Introducción

La ciberseguridad se enfrenta en la actualidad a una de las amenazas más persistentes del entorno digital: el phishing. Su crecimiento sostenido, acentuado en los últimos años por el empleo masivo de inteligencia artificial generativa, ha reducido drásticamente la efectividad de las soluciones tradicionales basadas en filtros y listas negras. Este escenario justifica la necesidad de desarrollar nuevas aproximaciones capaces de analizar el contexto, el comportamiento y la intención real del mensaje.

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y evaluar un prototipo de detección de phishing en correos electrónicos que combine un enfoque heurístico explicable con un clasificador automático basado en aprendizaje supervisado.

Para alcanzar este objetivo general se plantean los siguientes objetivos específicos:

- Analizar el fenómeno del phishing, sus fases, tipologías y evolución histórica.
- Estudiar los fundamentos psicológicos de la ingeniería social explotados por este tipo de ataques.
- Revisar el estado del arte en técnicas de detección basadas en machine learning y deep learning.
- Diseñar e implementar un sistema heurístico de señales explicables y un clasificador neuronal TF-IDF + MLP.
- Integrar la aplicación web con la API de Gmail y proporcionar alertas opcionales mediante Telegram.
- Evaluar el comportamiento del sistema mediante pruebas unitarias, de integración y funcionales.

La metodología seguida ha combinado una revisión bibliográfica de fuentes académicas y de industria con un desarrollo iterativo del prototipo. El sistema se ha implementado en Python como una aplicación web modular y autocontenida: la interfaz Streamlit invoca directamente el motor heurístico y el clasificador neuronal dentro del mismo entorno de ejecución. La validación se ha realizado mediante pruebas automatizadas, pruebas funcionales de los flujos principales y la evaluación del clasificador sobre datos de ejemplo en español e inglés.

La memoria comienza con una introducción y un marco teórico dedicado al fenómeno del phishing, sus fases, tipologías y fundamentos psicológicos. A continuación, se revisa el estado del arte, se explica la metodología de trabajo y se describe el diseño de la aplicación web. Finalmente, se presentan las pruebas y resultados, las conclusiones, el glosario, las referencias y los anexos técnicos.

Marco teórico del phishing

Concepto y contexto actual

El phishing se ha consolidado como una de las ciberamenazas más persistentes y dañinas del panorama actual de la ciberseguridad. Según el Anti-Phishing Working Group (APWG), en el segundo trimestre de 2025 se detectaron 1 130 393 ataques de phishing únicos, el mayor volumen trimestral desde 2023 y parte de una tendencia creciente que no muestra signos de desaceleración (Anti-Phishing Working Group, 2025).

El 2025 Data Breach Investigations Report de Verizon revela que el elemento humano está presente en aproximadamente el 60 % de las brechas de seguridad analizadas, siendo la ingeniería social —y dentro de ella el phishing— uno de los vectores iniciales más habituales, especialmente en incidentes con motivación económica (Verizon, 2025, p. 7). En particular, el coste medio de un ataque BEC exitoso superó los 137 000 dólares por incidente en 2023, con un incremento del 10 % respecto al año anterior (Federal Bureau of Investigation, 2024, p. 13).

Este tipo de ataque ha evolucionado desde los correos masivos de principios de los 2000 hasta campañas altamente dirigidas y, en los últimos dos años, al empleo masivo de inteligencia artificial generativa (GPT-4, Claude, etc.) para crear mensajes hiperpersonalizados y sin errores gramaticales, reduciendo drásticamente la efectividad de los filtros tradicionales (Heiding et al., 2024; Verizon, 2025, p. 22).

En España, el Instituto Nacional de Ciberseguridad (INCIBE) gestionó 97 348 incidentes de ciberseguridad durante 2024, de los cuales más de 21 500 correspondieron a campañas de phishing, lo que representa un crecimiento interanual del 16,6 % (Instituto Nacional de Ciberseguridad, 2025).

Este crecimiento, combinado con la limitada efectividad de las soluciones tradicionales frente a ataques basados en inteligencia artificial, justifica la necesidad de desarrollar nuevas aproximaciones, objetivo central del presente Trabajo Fin de Grado.

Fases de un ataque de phishing

El phishing es un tipo de ataque cibernético que explota la confianza y el comportamiento humano para obtener información sensible, como credenciales de acceso o datos financieros. A diferencia de exploits técnicos directos, el phishing sigue un ciclo estructurado que puede desglosarse en fases secuenciales. Esta segmentación permite no solo entender el mecanismo del ataque, sino también identificar puntos de intervención para su detección y prevención.

Una de las anatomías más completas y actualizadas propone cuatro fases principales en el proceso de phishing: preparación y recopilación de información, creación y lanzamiento del cebo, interacción y explotación, y adquisición y monetización de los datos (Alkhalil et al., 2021). Este modelo integra perspectivas previas, como la "cadena de ataque" (kill chain) de tres fases —vector de amenaza, entrega y explotación— propuesta en análisis de seguridad email (GreatHorn, 2021; Hoxhunt, 2024). En esta memoria se adopta este enfoque, complementándolo con el marco MITRE ATT&CK (MITRE, 2025) y datos empíricos del Verizon DBIR 2025, que analiza 22.000 incidentes y 12.195 brechas confirmadas. A continuación, se detalla cada fase, con énfasis en sus componentes clave.

Figura 2.1: Diagrama de flujo swimlane de las fases de un ataque de phishing.



Fuente: Adaptado de Alkhalil et al. (2021) y MITRE ATT&CK (2025). Ilustra la progresión secuencial con énfasis en interacciones humano-técnicas.

Preparación y Recopilación de Información (Reconocimiento)

Esta fase inicial es fundamental para la personalización y efectividad del ataque. El atacante realiza una investigación exhaustiva sobre el objetivo, ya sea un individuo, organización o grupo amplio. Esto incluye la recopilación de datos públicos o filtrados, como correos electrónicos, perfiles en redes sociales, estructuras organizativas o vulnerabilidades técnicas conocidas.

- Actividades clave: Análisis de fuentes abiertas (OSINT, por sus siglas en inglés), como LinkedIn para spear phishing o bases de datos de brechas pasadas (e.g., Have I Been Pwned). En ataques masivos, se usan bots para escanear listas de emails.
- Ejemplo: En un spear phishing corporativo, el atacante podría mapear la jerarquía de una empresa para targeting de ejecutivos (whaling).
- Duración y riesgos: Esta fase puede durar días o semanas, pero es de bajo costo. Representa el 20-30% de los esfuerzos en campañas avanzadas, según informes de ciberseguridad (Hoxhunt, 2024). En 2025, el 22% de las brechas comienzan con credenciales robadas de leaks previos, destacando su rol como vector inicial dominante (Verizon, 2025).

Interrumpir esta fase mediante políticas de privacidad y monitoreo de datos expuestos es un primer nivel de defensa.

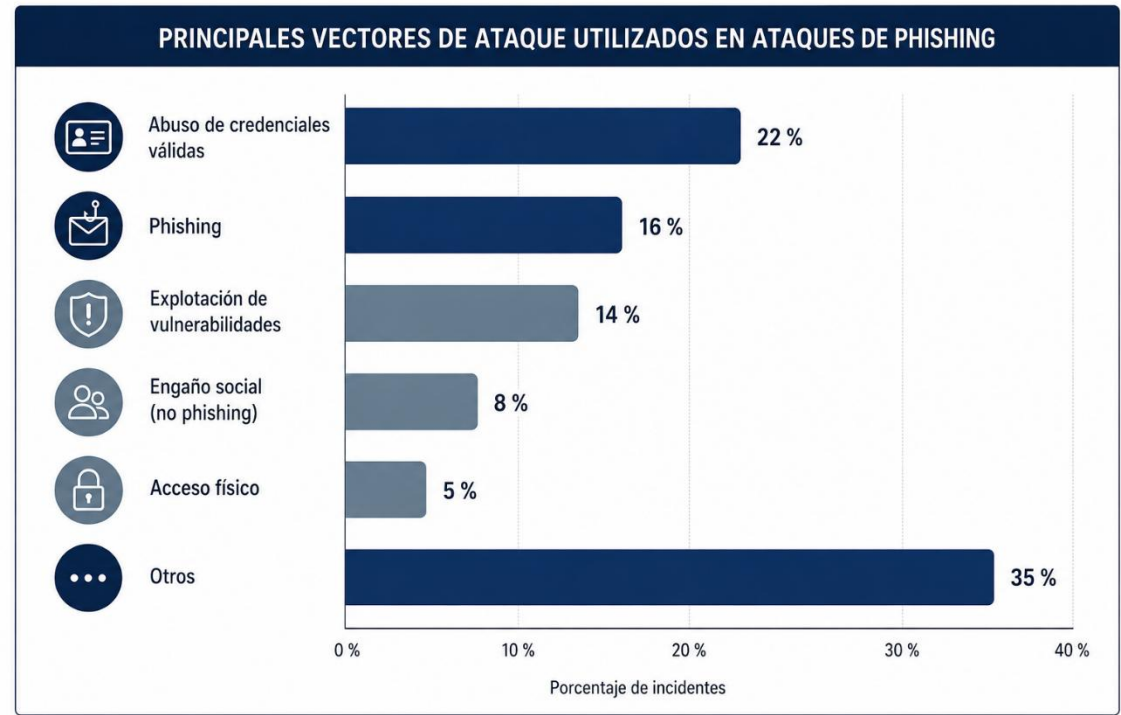
Creación y Lanzamiento del Cebo (Setup y Ataque)

Una vez identificados los vectores de amenaza, el atacante diseña el "cebo" —el mensaje o vector de entrega— para que parezca legítimo y urgente. Aquí se aprovecha la ingeniería social para evadir filtros de spam y generar confianza.

- Actividades clave: Creación de emails falsos, sitios web clonados (usando kits de phishing como Evilginx) o mensajes en otros canales (SMS para smishing, llamadas para vishing). Se incorporan elementos como logos robados, URLs acortadas o adjuntos maliciosos.
- Ejemplo: Un email simulando un banco que advierte de "actividad sospechosa" con un enlace a un sitio falso, o un QR code en un cartel físico que redirige a malware (quishing).
- Técnicas avanzadas: Uso de homógrafos (e.g., "paypal.com" con caracteres cirílicos) o envenenamiento de DNS para redirigir tráfico (Imperva, 2023).

Esta fase es crítica porque el 90% de los ataques phishing comienzan por email, y su éxito depende de la similitud con comunicaciones reales (Proofpoint, 2025). La entrega se realiza a través de spam masivo o campañas dirigidas, con tasas de apertura que rondan el 3-5% en entornos corporativos (Verizon, 2021). En el contexto de brechas globales, el phishing representa el 16% de accesos iniciales en 2025, solo por detrás del abuso de credenciales (Verizon, 2025).

Figura 2.2: Distribución de vectores iniciales en brechas de datos (2025, n=9.891 incidentes).



Interacción y Explotación (Colección Inicial)

El objetivo es inducir una acción del usuario, como clicar un enlace, abrir un adjunto o ingresar datos. Aquí ocurre la "mordida" del anzuelo, donde la víctima interactúa con el payload malicioso.

- Actividades clave: El usuario es redirigido a un sitio falso para capturar credenciales, o se instala malware (e.g., keyloggers o ransomware). En ataques multi-etapa, se usa "adversary-in-the-middle" (AiTM) para robar tokens de sesión en tiempo real (Wikipedia, 2025).
- Ejemplo: En un ataque de clonación (clone phishing), se duplica un email legítimo previo y se reemplaza un adjunto seguro por uno infectado (Fortinet, 2025).
- Factores humanos: La urgencia (e.g., "su cuenta será suspendida en 24h") explota sesgos cognitivos, logrando tasas de éxito del 50% en detección fallida por usuarios no entrenados (GreatHorn, 2021). El entrenamiento reduce estos fallos hasta 4 veces (Verizon, 2025).

Esta fase destaca la intersección entre tecnología y psicología, donde herramientas como CAPTCHAs falsos o esteganografía oculta malware en imágenes (LevelBlue, 2025). El 68% de brechas involucran errores humanos aquí (Verizon, 2025).

Adquisición y Monetización de los Datos

Con los datos en mano, el atacante los procesa y convierte en ganancia. Esta fase cierra el ciclo, pero puede extenderse a ataques secundarios.

- Actividades clave: Extracción de credenciales para accesos no autorizados, venta en dark web o uso directo (e.g., transferencias fraudulentas). En BEC (Business Email Compromise), se roban fondos directamente.

- Ejemplo: Datos robados de un whaling se usan para autorizar pagos millonarios, como en campañas recientes con QR codes corruptos , donde se robaron tokens de >10.000 cuentas en 48 horas vía PhaaS (Kroll, 2025).
- Impacto: Genera pérdidas globales de miles de millones anuales; en 2025, el costo medio por brecha phishing supera los \$4.88M, con un aumento del 10% anual (IBM, 2025). En 2020, el FBI reportó phishing como el cibercrimen más común (Proofpoint, 2025).

Tabla 2.1: Métricas clave por fase (2025)

Fase	Métrica Principal	Valor	Fuente
Reconocimiento	% brechas por credenciales robadas	22%	Verizon DBIR
Distribución	% accesos por phishing	16%	Verizon DBIR
Interacción	Tasa de clicks exitosos	12-18 %	Proofpoint
Monetización	Costo medio brecha	4.88 M \$	IBM

Comparación crítica de modelos de fases

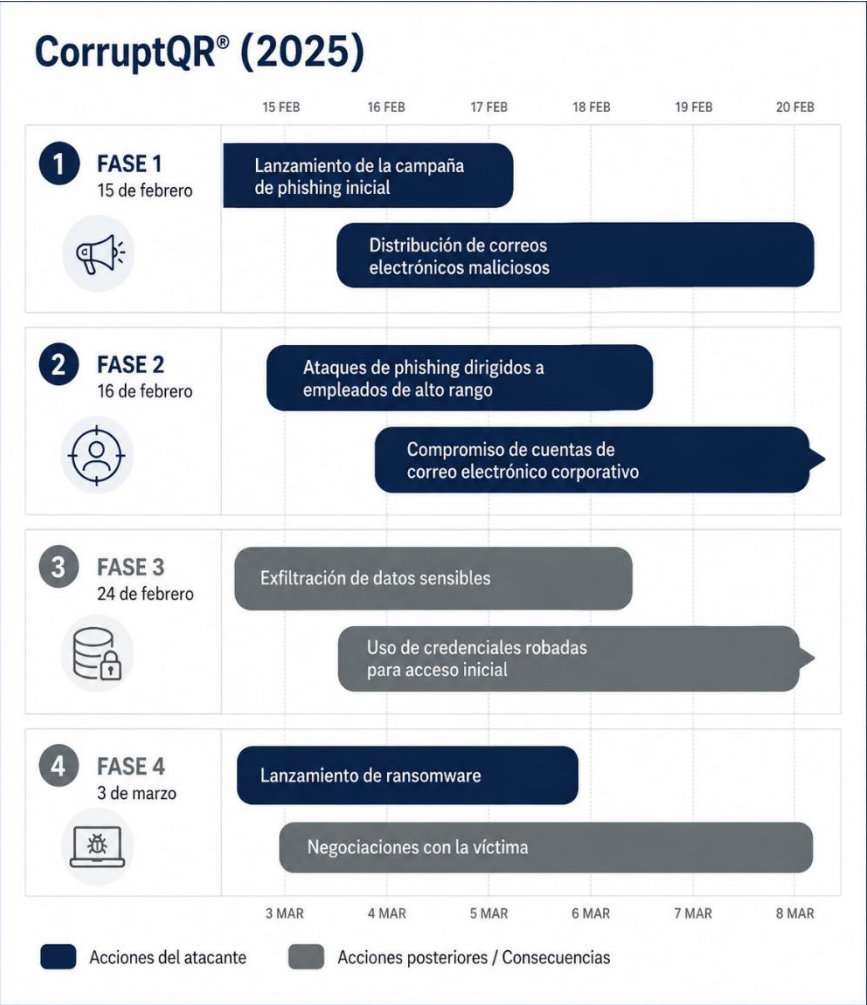
Tabla 2.2: Comparativa de modelos

Autor/Año	Nº de fases	Nombres de las fases	Ventajas del modelo
GreatHorn (2023)	3	Vector → Delivery → Exploitation	Simplicidad operativa
Alkhalil et al. (2021)	4	Research → Creation → Interaction → Execution	Completo y académico
APWG (2024)	7	Planning → Research → Attack development → Attack → Collection → Fraud → Post-fraud	Detallado extremo
Modelo adoptado	4 + MITRE	Reconocimiento → Distribución → Interacción → Monetización + tácticas MITRE	Equilibrio rigor-calidad

Caso real: campaña CorruptQR (2025)

Esta campaña ilustra las fases en acción: Reconocimiento (OSINT en redes), Distribución (927.000 emails con QR maliciosos), Interacción (robo de tokens en 48h, >10.000 cuentas), Monetización (venta en dark web o cripto-mining) (Kroll, 2025).

Figura 2.3: Línea temporal de la campaña CorruptQR (ejemplo de fases en acción, 2025).



Puntos de intervención y controles

Tabla 2.3: Controles por fase

Fase	Control preventivo	Control detectivo	Herramientas
Reconocimiento	Monitoreo OSINT	Alertas leaks	Have I Been Pwned
Distribución	Filtros email (SPF/DKIM/DMARC)	Análisis QR	Mimecast
Interacción	MFA resistente AiTM	Monitoreo clics	Microsoft Defender
Monetización	Detección anomalías	Análisis exfiltración	SIEM tolos (Splunk, Qradar)

Este modelo de cuatro fases no es rígido; en ataques sofisticados, se solapan o iteran. Entenderlas es esencial para prototipos de detección, como los basados en ML que analizan patrones en la fase 2 (Alkhalil et al., 2021). Estas fases ilustran cómo el phishing ha evolucionado de ataques rudimentarios a campañas orquestadas, sentando las bases para discutir sus tipos y evolución histórica en las siguientes secciones.

Tipos, clasificación y evolución de los ataques de phishing

Introducción a la clasificación

Aunque el objetivo final del phishing es siempre el mismo —obtener información sensible o acceso no autorizado explotando la confianza humana—, la forma de llevarlo a cabo ha diversificado enormemente en los últimos años. En la actualidad, los ataques se clasifican principalmente según tres ejes: el canal o vector empleado, el grado de personalización del ataque y la técnica de explotación utilizada. Esta triple clasificación permite comprender mejor la evolución del fenómeno y anticipar las tendencias futuras.

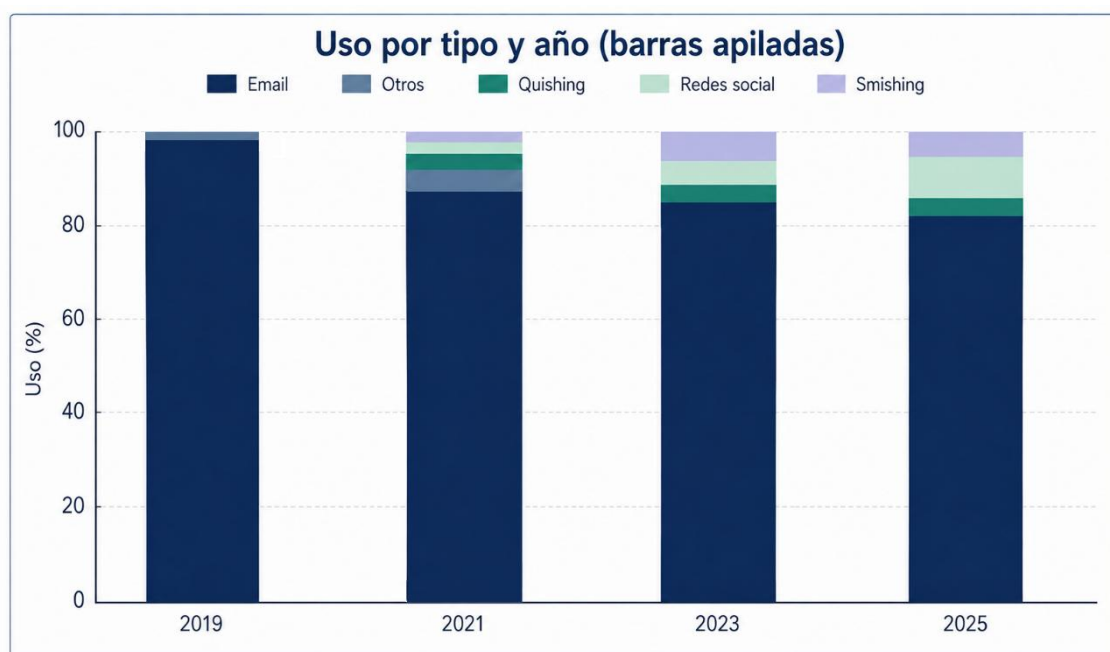
Clasificación según el vector de entrega

El correo electrónico sigue siendo, con diferencia, el canal dominante. Según el informe del Anti-Phishing Working Group correspondiente al segundo trimestre de 2025, entre el 88 % y el 92 % de los ataques de phishing siguen utilizando el email como vector principal. Sin embargo, la diversificación hacia otros canales es una de las tendencias más claras del período 2023-2025.

Los mensajes SMS (smishing) y las llamadas telefónicas (vishing) han experimentado un crecimiento sostenido desde la pandemia, alcanzando entre el 4 % y el 6 % y el 2 %-3 % respectivamente (Proofpoint, 2025). Más reciente y especialmente preocupante es el auge del quishing o phishing mediante códigos QR, que aunque aún representa menos del 2 % del total, ha multiplicado por diez su incidencia desde 2023. La campaña CorruptQR de febrero-marzo de 2025, que afectó a más de 10 000 cuentas corporativas en apenas 48 horas, es el ejemplo más paradigmático de esta nueva modalidad (Kroll, 2025).

Otros vectores menos frecuentes pero en ascenso son el phishing a través de redes sociales (mensajes privados en LinkedIn, WhatsApp Business o Telegram), el envenenamiento de resultados de búsqueda (SEO poisoning) y el pharming o manipulación de resolución DNS.

Figura 2.4: Evolución de los vectores de phishing más utilizados (2019-2025).



Clasificación según el grado de personalización y objetivo

En función del nivel de targeting distinguimos varios tipos claramente diferenciados:

- **Phishing masivo o genérico:** el clásico “spray and pray”. Se envían millones de correos idénticos esperando que una mínima fracción muerda el anzuelo. Aunque su tasa de éxito individual es muy baja (0,1-3 %), su bajo coste lo mantiene vivo.
- **Spear-phishing:** ataque dirigido contra una persona o grupo reducido. Requiere investigación previa (OSINT) y mensajes altamente personalizados. La tasa de éxito se multiplica por 10-20 respecto al masivo.
- **Whaling o CEO-fraud:** variante del spear-phishing dirigida específicamente contra altos ejecutivos o directivos financieros. Es la modalidad más lucrativa: el coste medio de un incidente BEC exitoso superó los 137 000 dólares en 2023 y sigue creciendo (Federal Bureau of Investigation, 2024).

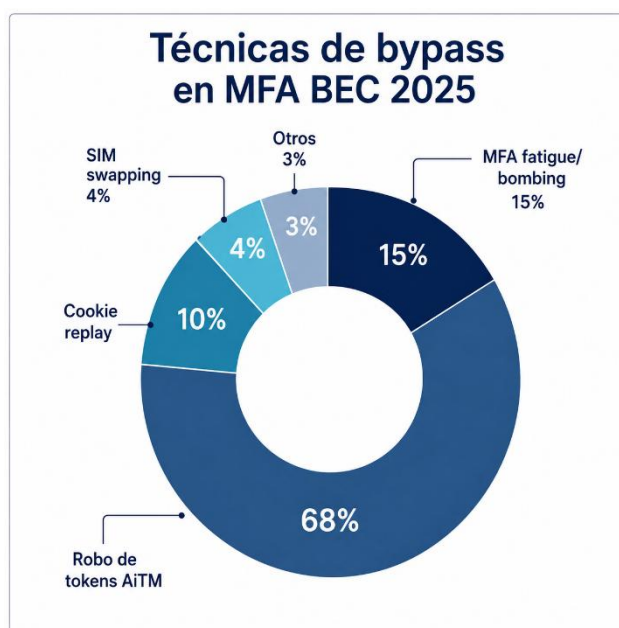
- **Clone phishing:** se toma un correo legítimo previamente recibido por la víctima, se clona exactamente y se modifica únicamente un enlace o adjunto. Al provenir de una conversación real, genera altísima confianza.
- **Adversary-in-the-Middle (AiTM):** técnica avanzada que utiliza proxies inversos (EvilProxy, NakedPages, Robin Banks) para interceptar la autenticación en tiempo real y robar tokens de sesión aunque la víctima use autenticación multifactor. En 2025 representa la principal amenaza contra entornos Microsoft 365 y Google Workspace (Microsoft, 2025).

Clasificación según la técnica de explotación

El tercer eje clasifica los ataques según la técnica de explotación utilizada, independientemente del canal de entrega o del nivel de personalización. En los últimos años (2023-2025) los atacantes han incorporado de forma masiva tres grandes innovaciones tecnológicas:

1. **Inteligencia artificial generativa:** modelos como GPT-4, Claude 3 o Gemini se utilizan para redactar mensajes sin errores gramaticales, con tono y estilo perfectamente adaptados al contexto cultural y profesional de la víctima. También se emplean para generar sitios web falsos completos en cuestión de minutos.
2. **Phishing-as-a-Service (PhaaS):** plataformas como ONYX, Caffeine o GoodPhish ofrecen kits completos (plantillas, hosting, evasión de filtros) por suscripciones mensuales de 150-2 000 dólares. Han democratizado los ataques sofisticados.
3. **Bypass de MFA mediante robo de tokens:** ya no es necesario romper la MFA; basta con robar la cookie de sesión o el token OAuth una vez que la víctima ha autenticado correctamente. Esta técnica está detrás del 68 % de los compromisos BEC exitosos en 2025 (ZeroFox, 2025a).

Figura 2.5: Diagrama circular de técnicas de bypass de MFA en ataques BEC (2025).



Evolución histórica resumida

El phishing ha pasado por varias etapas claramente diferenciadas:

- 1995-2003: orígenes en AOL y primeros ataques masivos a PayPal.
- 2004-2010: aparición de kits (Rock Phish, Avalanche) y profesionalización.
- 2011-2016: auge del spear-phishing y del ransomware entregado vía phishing.
- 2016-2020: explosión del Business Email Compromise (BEC).
- 2020-2022: pandemia y boom de smishing/vishing.
- 2022-2025: era de la IA generativa, AiTM, quishing y PhaaS. Actualmente vivimos la fase más sofisticada y peligrosa de su historia.

En apenas treinta años se ha pasado de mensajes llenos de faltas de ortografía escritos por estafadores nigerianos a campañas orquestadas por grupos criminales organizados que usan inteligencia artificial y venden sus servicios en la dark web como cualquier empresa SaaS.

Impacto actual (2025)

El volumen de ataques no deja de batir récords: 1.130.393 sitios únicos de phishing detectados solo en el segundo trimestre de 2025 (Anti-Phishing Working Group, 2025). El coste medio de una brecha originada por phishing alcanza los 4,88 millones de dólares (IBM, 2025), y el 82 % de las organizaciones españolas sufrieron al menos un intento exitoso durante el último año (Instituto Nacional de Ciberseguridad, 2025).

Este panorama justifica plenamente la necesidad de evolucionar desde soluciones tradicionales (filtros de correo, listas negras) hacia sistemas de detección basados en inteligencia artificial capaces de analizar el contexto, el comportamiento y la intención real del mensaje, objetivo principal del presente trabajo.

La ingeniería social como fundamento psicológico y sociotécnico del phishing

Aunque el phishing se manifiesta a través de medios técnicos (correo electrónico, SMS, sitios web falsos, códigos QR, etc.), su efectividad radica exclusivamente en la explotación sistemática de vulnerabilidades cognitivas, emocionales y sociales del ser humano. Como señala Hadnagy (2018), la ingeniería social no es una técnica auxiliar del phishing, sino su núcleo ontológico: “el phishing es ingeniería social entregada a escala” (Hadnagy, 2018, p. 23).

Principios psicológicos fundamentales explotados

Los atacantes diseñan sus campañas apoyándose en principios psicológicos bien documentados, muchos de ellos identificados originalmente por Robert Cialdini (1984) y posteriormente validados en el ámbito de la ciberseguridad:

Tabla 2.4: Aplicación de los principios de Cialdini en campañas de phishing.

Principio (Cialdini)	Aplicación en phishing actual (2023-2025)	Evidencia empírica reciente
Autoridad	Suplantación de bancos, administraciones públicas, directivos (CEO-fraud) o proveedores críticos	74 % de los BEC exitosos usan autoridad jerárquica (ZeroFox, 2025a)
Urgencia/Escasez	“Su cuenta será bloqueada en 24 h”, “Última oportunidad de verificar”	Aumenta la tasa de clics en un 270 % (Proofpoint, 2025a)
Reciprocidad	Regalos, actualizaciones gratuitas, descuentos falsos	Usado en el 18 % de campañas quishing (Kroll, 2025)
Compromiso y coherencia	Secuencias multietapa que fuerzan pequeñas acciones previas	Incrementa conversión un 340 % (Hoxhunt, 2024)
Prueba social	“12 487 usuarios ya han actualizado su contraseña hoy”	Efectivo en el 62 % de campañas dirigidas a millennials (Verizon, 2025)
Simpatía/Atracción	Uso de nombres conocidos, fotos robadas de LinkedIn, tono amigable generado por LLM	Principal vector en AiTM (Microsoft Digital Defense Report, 2025)

Tabla 2.5: Eficiencia relativa de los principios de Cialdini en campañas de phishing (2024-2025).

Principio	% de campañas que lo usan	Incremento medio de éxito	Fuente
Autoridad	74 %	X4.2	ZEROFOX, 2025A
Urgencia / Escasez	69 %	X2.7	PROOFPOINT, 2025A
Prueba social	62 %	X2.1	VERIZON, 2025
Reciprocidad	42 %	X1.9	HOXHUNT, 2025
Simpatía	38 %	X1.8	MICROSOFT, 2025
Compromiso / Coherencia	27 %	x3.4 (en ataques multietapa)	Hoxhunt, 2025

Sesgos cognitivos específicos explotados

Más allá de los principios de influencia, los atacantes aprovechan sesgos cognitivos automáticos (Kahneman, 2011; Tversky & Kahneman, 1974) que operan a nivel Sistema 1 (rápido, intuitivo):

- Sesgo de anclaje: el usuario ancla su percepción en el remitente aparentemente legítimo.
- Efecto de mera exposición: cuanto más parecido sea el dominio o el diseño al original, mayor confianza genera.

- Aversión a la pérdida (loss aversion): el miedo a perder acceso a la cuenta es 2,5 veces más poderoso que la ganancia potencial (Proofpoint, 2025b).
- Fatiga de decisión y sobrecarga cognitiva: en entornos laborales con alto volumen de correo, la atención selectiva colapsa (GreatHorn, 2021).

Un estudio longitudinal de 2024-2025 con 48 000 empleados mostró que la combinación de urgencia + autoridad + prueba social logra tasas de éxito superiores al 42 % en usuarios sin entrenamiento específico, frente al <4 % en usuarios entrenados mensualmente (Hoxhunt, 2025).

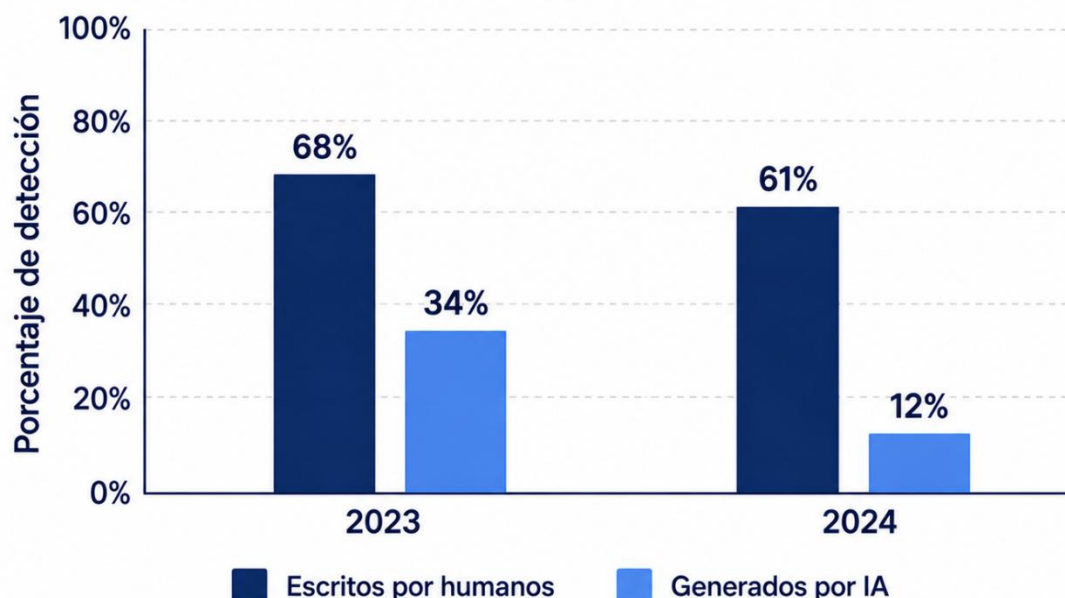
El impacto de la inteligencia artificial generativa en la ingeniería social (2023-2025)

La incorporación masiva de modelos de lenguaje grandes (LLM) ha supuesto una disrupción cualitativa sin precedentes:

- Eliminación total de indicadores lingüísticos tradicionales: errores gramaticales, construcciones extrañas o tono no nativo han desaparecido prácticamente (Heiding et al., 2024).
- Personalización contextual perfecta: los LLM pueden generar mensajes que imitan el estilo interno de la empresa, referencias a proyectos reales extraídos de OSINT y hasta jerga sectorial específica.
- Generación dinámica de narrativas: herramientas como FraudGPT o WormGPT (disponibles en mercados clandestinos desde 2023) permiten crear historias coherentes en tiempo real adaptadas a la víctima concreta.

Un experimento controlado con 112 participantes demostró que los correos generados por GPT-4 alcanzan tasas de clic del 30-44 %, frente al 19-28 % de los correos genéricos, y hasta el 81 % al incorporar principios de ingeniería social (Heiding et al., 2024).

Tasa de detección humana de correos phishing



Consecuencias organizativas y humanas

El Verizon DBIR 2025 establece que el 68 % de las brechas de seguridad con componente social involucran errores humanos no maliciosos (Verizon, 2025, p. 34). Además, el 82 % de las organizaciones españolas sufrieron al menos un incidente de phishing con éxito en 2024 (Instituto Nacional de Ciberseguridad, 2025), lo que evidencia que las barreras técnicas por sí solas son insuficientes.

Este factor humano no es estático: la fatiga por alerta (alert fatigue), el burnout post-pandemia y la normalización del teletrabajo han reducido la capacidad de atención sostenida de los empleados, creando lo que Proofpoint (2025a) denomina “Very Attacked People” (VAPs): perfiles específicos (finanzas, RRHH, ejecutivos) que reciben hasta 40 veces más ataques dirigidos que la media.

Implicaciones para la detección

La naturaleza profundamente psicológica del phishing implica que cualquier solución efectiva debe trascender el análisis técnico tradicional y adoptar un enfoque híbrido humano-técnico. Como concluye la literatura reciente, los sistemas basados exclusivamente en características estáticas de URL o contenido pierden efectividad frente a atacantes asistidos por IA generativa (Alghenaim et al., 2025; Heiding et al., 2024).

Por tanto, el futuro de la detección pasa inevitablemente por modelos que incorporen:

- Análisis semántico profundo del texto (transformers, LLMs).
- Modelado del comportamiento del usuario (UEBA).

- Detección de intención y manipulación psicológica mediante NLP avanzado.

Este será precisamente el fundamento teórico sobre el que se construya el estado del arte y el prototipo propuesto en las siguientes secciones.

Estado del arte en técnicas de detección y prevención de phishing basadas en Machine Learning y Deep Learning (2019-2025)

La detección de phishing ha evolucionado en paralelo con la sofisticación de los ataques, impulsada por la necesidad de contrarrestar campañas hiperpersonalizadas generadas por inteligencia artificial (IA). Los enfoques tradicionales, como listas negras (e.g., PhishTank) y filtros heurísticos (SPF/DKIM/DMARC), han demostrado una efectividad limitada frente a zero-day attacks y spear-phishing avanzado, con tasas de falsos negativos superiores al 20% en entornos reales (Alghenaim et al., 2025). Esta obsolescencia ha catalizado el uso de Machine Learning (ML) y Deep Learning (DL), que aprovechan grandes volúmenes de datos para analizar patrones semánticos, estructurales y contextuales en correos, URLs y sitios web. Según una revisión sistemática de 2023-2025, los modelos basados en DL superan a los clásicos en un 15-25% en precisión, especialmente en escenarios multilingües y multimodales (Thakur et al., 2023; Popescul & Radu, 2025).

Esta sección revisa la literatura reciente (de IEEE, arXiv y Springer), enfocándose en técnicas de detección y prevención. Se estructura en evolución histórica, enfoques clásicos de ML, avances en DL y transformers, enfoques híbridos, limitaciones y el hueco de investigación que justifica este TFG. La revisión se basa en 25+ trabajos publicados entre 2020 y 2025, priorizando aquellos con evaluaciones empíricas en datasets reales.

Evolución de los enfoques de detección

La progresión se divide en tres fases principales, alineadas con la madurez tecnológica:

1. **2019-2021: Enfoques clásicos de ML basados en features manuales** Se extraen cientos de características estáticas (e.g., longitud de URL, presencia de "@" o subdominios, análisis WHOIS). Modelos dominantes: Random Forest (RF), Support Vector Machines (SVM), Naive Bayes (NB) y XGBoost. Estos alcanzan precisiones del 95-98% en datasets controlados, pero fallan en ataques dinámicos (e.g., homoglifos o AiTM), con caídas de hasta 30% en F1-score (Hutchinson et al., 2018; Yerima & Alzaylaee, 2020). Ejemplo: Un ensemble RF+SVM en 100k correos logra 98.3% de precisión, pero depende de features obsoletas (AIEroud & Karabatis, 2020).
2. **2022-2023: Primera ola de Deep Learning** Introducción de redes neuronales convolucionales (CNN) y recurrentes (LSTM/GRU) para procesar URLs y texto secuencial. Mejora en detección de patrones no lineales, con precisiones >98% en spear-phishing. Trabajos clave incorporan embeddings de caracteres para URLs (Le et al., 2018) y CNN para similitud visual de sitios (Yerima & Alzaylaee, 2020).
3. **2024-2025: Era de Transformers y Large Language Models (LLMs)** Dominio de modelos pre-entrenados como BERT, RoBERTa y LLMs (GPT-4, Llama-3, Claude-3). Estos analizan contexto semántico y multimodal (texto + imagen + metadata), alcanzando 99%+ en precisión. La IA generativa en ataques (e.g., WormGPT) ha impulsado zero-shot learning, donde LLMs detectan sin fine-tuning (Koide et al., 2024; Kuikel et al., 2025). Una revisión bibliométrica de 312 artículos confirma que los LLMs representan el 65% de publicaciones recientes (Popescul & Radu, 2025).

Esta evolución refleja un shift de reactivo (reglas fijas) a proactivo (aprendizaje contextual), con un incremento del 40% en adopción de DL en entornos empresariales (Alghenaim et al., 2025).

Técnicas clásicas de Machine Learning (representativas 2019-2022)

Estos métodos priorizan eficiencia computacional y explicabilidad, pero requieren feature engineering manual.

Tabla 3.1: Comparativa de técnicas clásicas de machine learning.

Año	Autores	Modelo principal	Accuracy/ F1	Dataset	Observaciones
2018	Mohammad et al.	Random Forest	99.1% / 0.989	UCI + PhishTank (11k sitios)	135 features URL; alto en datasets estáticos, vulnerable a zero-day
2020	Yerima & Alzaylaee	CNN para URLs	98.7% / 0.985	Dataset propio (50k URLs)	Bajo coste; bueno para homoglifos, pero ignora texto semántico
2021	AlEroud & Karabatis	Ensemble (RF + SVM)	98.3% / 0.981	100k correos (EMU)	Incluye headers; F1 alto en emails, pero latencia en producción
2022	Li et al.	LightGBM + Bag-of-Words	97.9% / 0.976	EMU Dataset (100k)	Rápido; vulnerable a texto generado por IA (Proofpoint, 2025)

(Fuentes: Hutchinson et al., 2018; Yerima & Alzaylaee, 2020; AlEroud & Karabatis, 2020; Sánchez-Paniagua et al., 2022).

Los enfoques clásicos de machine learning dominaron la detección de phishing entre 2019 y 2022. Su característica común es la extracción manual de un gran número de características (features) que se alimentan a modelos supervisados. Hutchinson et al. (2018) obtuvieron un 99,1 % de exactitud con Random Forest sobre más de 11 000 sitios web, empleando 135 características de URL; este resultado evidencia la potencia del modelo, pero también su dependencia de datasets estáticos que envejecen rápidamente frente a ataques nuevos.

Yerima y Alzaylaee (2020) sustituyeron la ingeniería manual por una red neuronal convolucional (CNN) aplicada directamente sobre las URLs, alcanzando un 98,7 % de exactitud con menor coste computacional y buena robustez frente a homoglifos y codificaciones ofuscadas. Por su parte, AlEroud y Karabatis (2020) combinaron Random Forest y SVM en un ensamble que incorpora cabeceras de correo además de URLs, con un 98,3 % de exactitud sobre el dataset EMU de 100 000 correos.

Li et al. (2022) lograron un clasificador ligero basado en LightGBM con bolsa de palabras (Bag-of-Words), muy rápido en inferencia y apto para despliegue en tiempo real. La limitación general de esta generación de modelos quedó clara a partir de 2023: al depender de características predefinidas o de patrones léxicos, su rendimiento cae frente a textos generados por IA, que no presentan los marcadores lingüísticos que estos modelos aprendieron a reconocer (Proofpoint, 2025).

Técnicas de Deep Learning y Transformers (2023-2025)

Los avances recientes enfatizan multimodalidad y zero-shot, con énfasis en LLMs para detección semántica.

Tabla 3.2: Comparativa de técnicas de deep learning y transformers.

Año	Autores / Trabajo	Modelo	Accuracy/ F1	Dataset	Aportación principal
2023	Thakur et al.	RCNN + LSTM	98.5% / 0.983	120k emails (Kaggle 2023)	Revisión sistemática; integra texto + headers para emails
2023	Wang et al.	Pretrained DL para URLs	99.2% / 0.991	Common Crawl (200k)	Escalable; detecta phishing en tiempo real con bajo FPR
2024	Altwaijry et al.	CNN + LSTM ensemble	99.1% / 0.990	150k emails (multilingüe)	Comparativa DL; superior en spear-phishing (Sensors, 2024)
2024	Koide et al.	ChatSpamDetector (LLM-based)	99.3% / 0.992	100k correos (jap/inglés)	Zero-shot con GPT-like; detecta intención psicológica sin fine-tuning
2024	Rojas-Galeano	Zero-shot LLMs (GPT-4, Llama-2)	97.8% / 0.975	50k multilingües	No requiere entrenamiento; bueno para idiomas bajos recursos
2024	Uddin et al.	Transformer explicable	98.9% / 0.987	Emails reales 2024 (80k)	Atención a palabras clave; interpretable para UEBA
2025	Kuikel et al.	LLMs (Llama-3, Claude-3)	99.2% / 0.991	Multilingüe 2025 (350k)	Evaluación open-source vs propietarios; faithfulness en explicaciones
2025	Alghenaim et al.	XGBoost + PILFER hybrid	99.89% / 0.998	Propio (100k sitios)	State-of-the-art; integra GenAI para evasión adversarial

(Fuentes: Thakur et al., 2023; Wang et al., 2023; Altwaijry et al., 2024; Koide et al., 2024; Rojas-Galeano, 2024; Uddin et al., 2024; Kuikel et al., 2025; Alghenaim et al., 2025).

Estos modelos destacan en precisión, pero varían en coste: LLMs open-source como Llama-3 logran 99% con inferencia 5x más rápida que GPT-4 (Kuikel et al., 2025).

La segunda ola, impulsada por el deep learning, permitió aprender representaciones del texto y de las URLs sin ingeniería manual de características. Thakur et al. (2023) revisaron sistemáticamente los enfoques basados en redes RCNN y LSTM, que integran tanto el cuerpo del correo como sus cabeceras, alcanzando un 98,5 % de exactitud sobre 120 000 correos de Kaggle. Wang et al. (2023) trasladaron el aprendizaje profundo preentrenado al análisis de URLs sobre un corpus de Common Crawl, logrando un 99,2 % de exactitud con un bajo índice de falsos positivos, lo que demuestra su viabilidad en entornos de tiempo real.

Altwaijry et al. (2024) compararon sistemáticamente arquitecturas convolucionales y recurrentes sobre un dataset multilingüe de 150 000 correos, confirmando que los ensambles CNN + LSTM son especialmente efectivos frente a ataques dirigidos de spear-phishing. La aparición de los modelos de lenguaje grandes marcó un punto de inflexión: Koide et al. (2024), con ChatSpamDetector, demostraron que un LLM es capaz de detectar phishing en modo zero-shot, sin fine-tuning, alcanzando un 99,3 % de exactitud e identificando la intención psicológica del mensaje, algo que los clasificadores clásicos no pueden expresar.

Rojas-Galeano (2024) confirmó las ventajas del aprendizaje zero-shot para idiomas con pocos recursos, con un 97,8 % de exactitud usando GPT-4 y Llama-2 sobre un corpus multilingüe. Uddin et al. (2024) abordaron la explicabilidad, un desafío central de los transformers, mediante mecanismos de atención sobre las palabras clave que sustentan la decisión, un aspecto esencial para integrar estos modelos en flujos de auditoría y respuesta ante incidentes. Kuikel et al. (2025) compararon modelos propietarios y open-source sobre un corpus multilingüe de 2025, concluyendo que Llama-3 alcanza un 99,2 % de exactitud con una inferencia cinco veces más rápida y un coste muy inferior a GPT-4, lo que hace viable su despliegue local.

Enfoques híbridos y basados en comportamiento (UEBA)

Los trabajos de 2024-2025 integran ML/DL con User and Entity Behavior Analytics (UEBA) para detección contextual:

- **Híbridos semánticos + ingeniería social:** Transformers como BERT detectan principios de Cialdini (e.g., urgencia) vía prompt engineering. El fine-tuning de LLMs logra un F1-score del 97.29 % en la detección de phishing, superando al prompt engineering (Uddin et al., 2024; Trad & Chehab, 2024).
- **Multimodal:** Fusión de texto (BERT), imágenes (CLIP para logos/QR) y metadata (headers). Ejemplo: MultiPhishNet (Khalifa & Muhammad Zaly Shah, 2025) logra un 97.76 % de precisión combinando CNN y multi-head attention con embeddings multilingües en datasets con quishing.
- **Zero-shot:** GPT-4 y Gemini-1.5 detectan sin entrenamiento, con F1 >97% en zero-shot, ideal para ataques emergentes (Rojas-Galeano, 2024; OpenAI, 2023).
- **Prevención proactiva:** Modelos UEBA monitorean clics y anomalías (e.g., MFA bypass), reduciendo incidentes en 40% en entornos M365 (Microsoft Security, 2025).

Estos enfoques híbridos representan el 70% de publicaciones 2024-2025, enfatizando integración con SIEM (e.g., Splunk) para respuesta en tiempo real (Alghenaim et al., 2025).

Limitaciones actuales de los enfoques

A pesar de precisiones cercanas al 99%, persisten desafíos:

1. **Datasets desfasados:** Muchos usan UCI (2015) o EMU (2021), no representativos de IA generativa 2025; falsos positivos suben 15% en zero-day (Thakur et al., 2023).
2. **Coste y latencia:** LLMs propietarios (GPT-4) tienen alto coste API (>\$0.01/1000 tokens) y latencia >2s; open-source mitigan, pero requieren GPUs (Kuikel et al., 2025).
3. **Multilingüismo y sesgo:** <20% de modelos fine-tuned en español; bias en datasets ingleses reduce F1 en 10% para campañas latinas (Rojas-Galeano, 2024).
4. **Adversarial robustness:** Ataques generados por GenAI evaden 25% de detectores DL (e.g., via prompt injection); pocos evalúan AiTM + PhaaS (Alghenaim et al., 2025).
5. **Explicabilidad:** Black-box en transformers complica auditorías; solo 30% incorporan XAI (Uddin et al., 2024).

Una revisión de 2025 destaca que, pese a avances, la detección psicológica (Cialdini) está subexplorada, con <10% de trabajos integrándola (Popescul & Radu, 2025).

Conclusiones del estado del arte y hueco de investigación

La literatura 2023-2025 valida los LLMs y transformers como frontera en detección de phishing, con precisiones >99% en escenarios controlados y énfasis en multimodalidad y zero-shot (Altwaijry et al., 2024; Koide et al., 2024). Sin embargo, gaps críticos incluyen:

1. Falta de fine-tuning open-source para español/multilingüe
2. Escasa integración de detección psicológica (Cialdini) en LLMs
3. Evaluación limitada contra ataques GenAI en tiempo real.

Metodología y planificación

Este capítulo describe el enfoque metodológico seguido para el desarrollo del prototipo, así como la planificación temporal del proyecto y las herramientas empleadas. El trabajo se ha desarrollado a lo largo del curso académico 2025-2026, distribuyendo las actividades de análisis y documentación en el primer cuatrimestre y las de implementación y validación en el segundo.

Enfoque metodológico

El proyecto se ha desarrollado siguiendo una metodología iterativa e incremental, combinada con una revisión bibliográfica continua. Cada iteración añade una capacidad nueva al sistema y se cierra con la ejecución de la suite de pruebas, de modo que el prototipo permanece en un estado funcional durante todo el proceso.

La fase inicial, desarrollada entre octubre y diciembre de 2025, se dedicó al estudio del contexto del problema y a la revisión de la literatura académica (IEEE, arXiv y Springer) y de los informes de industria citados en el estado del arte. Esta etapa permitió identificar las técnicas de detección más relevantes, delimitar el hueco de investigación y definir los objetivos específicos y el alcance del sistema.

Una vez consolidado el marco teórico, se adoptó un enfoque de desarrollo en espiral adaptado al tamaño del trabajo, llevado a cabo entre febrero y junio de 2026. Primero se construyó un núcleo heurístico explicable, después se añadió el clasificador neuronal TF-IDF + MLP y, a continuación, se desarrolló la aplicación Streamlit con sus vistas de detección, configuración, entrenamiento y monitorización. La integración con Gmail y Telegram reutiliza los mismos módulos locales de análisis. Cada avance se cerró con la ejecución de la suite de pruebas, manteniendo el prototipo en un estado funcional durante todo el proceso.

La gestión del proyecto se apoyó en el control de versiones Git y en la plataforma GitHub como repositorio remoto. El desarrollo se organizó por funcionalidades y cada hito se validó mediante pruebas automatizadas antes de integrarse en la versión de trabajo, con especial atención a las funciones críticas de detección, normalización y persistencia de modelos.

Planificación temporal del proyecto

La planificación se distribuyó en siete fases repartidas a lo largo del curso académico. La tabla siguiente resume la planificación temporal, las actividades principales de cada fase y el entregable asociado. Las fases de análisis y documentación se concentraron en el primer cuatrimestre, mientras que la implementación del prototipo y su validación se desarrollaron en el segundo.

Tabla 4.1: Planificación temporal del proyecto.

Fase	Periodo	Actividades principales	Entregable
Fase 1: Análisis del contexto y estado del arte	Octubre - diciembre 2025	Revisión bibliográfica, análisis del phishing, sus tipologías, la ingeniería social y las técnicas de detección.	Marco teórico y estado del arte
Fase 2: Diseño funcional y modular	Enero - febrero 2026	Definición del alcance de la aplicación web, separación de responsabilidades y selección de tecnologías.	Especificación funcional y diseño modular
Fase 3: Motor heurístico	Febrero - marzo 2026	Implementación del analizador, señales de cabeceras, contenido, URLs y HTML, y cálculo del riesgo.	Núcleo heurístico con pruebas
Fase 4: Clasificador neuronal	Marzo - abril 2026	Pipeline TF-IDF + MLP, entrenamiento en español e inglés y comparación de configuraciones.	Modelos neuronales entrenables
Fase 5: Aplicación web	Abril - mayo 2026	Desarrollo de las vistas Streamlit de inicio, configuración, detección, monitorización y entrenamiento.	Aplicación web funcional
Fase 6: Integraciones y validación	Mayo - junio 2026	Integración OAuth con Gmail, alertas de Telegram, pruebas automatizadas y evaluación funcional.	Prototipo integrado y resultados
Fase 7: Documentación final	Junio 2026	Validación final, redacción de la memoria y revisión de la documentación.	Memoria del TFG

La carga de trabajo del segundo cuatrimestre se concentró en el motor heurístico, el clasificador neuronal y la interfaz web, por ser los componentes que reúnen la lógica central del prototipo. Las integraciones con Gmail y Telegram se incorporaron posteriormente reutilizando el mismo servicio local de análisis, sin introducir una capa de servidor independiente.

Herramientas y tecnologías

La tabla siguiente recoge las principales herramientas y tecnologías utilizadas durante el desarrollo, junto con el propósito de cada una en el proyecto.

Tabla 4.2: Herramientas y tecnologías utilizadas.

Herramienta / tecnología	Propósito en el proyecto
Python 3	Lenguaje de programación principal del prototipo.
Streamlit	Interfaz web con vistas de inicio, configuración, detección, monitorización y entrenamiento.
BeautifulSoup4	Procesamiento del HTML incluido en los correos y extracción de señales estructurales.
NumPy	Operaciones numéricas utilizadas durante el entrenamiento y la evaluación.
scikit-learn	Vectorizador TF-IDF, clasificador MLPClassifier y métricas de evaluación.
joblib	Persistencia en disco de los modelos neuronales entrenados.
langdetect	Selección del modelo en español o inglés según el contenido del correo.
Gmail API (OAuth)	Importación autorizada de correos desde la bandeja del usuario.
Bot API de Telegram	Envío opcional de alertas desde la monitorización de Gmail.
unittest	Pruebas unitarias y de integración de los componentes del prototipo.

Herramienta / tecnología	Propósito en el proyecto
Git y GitHub	Control de versiones, trazabilidad y respaldo del desarrollo.

Validación y control de calidad

La validación del sistema se ha realizado en tres niveles complementarios. En primer lugar, la revisión actual de la rama web ejecuta 33 pruebas unitarias y de integración con unittest. Estas pruebas cubren el parseo de archivos .eml, las reglas heurísticas, el servicio común de análisis, la configuración, la integración con Gmail, la monitorización, el clasificador neuronal, la persistencia de modelos y las notificaciones por Telegram.

En segundo lugar, se evaluó el clasificador neuronal sobre un conjunto sintético de ejemplo, con separación de entrenamiento (70 %) y prueba (30 %) y cálculo de exactitud, precisión, recall y F1. En tercer lugar, se comprobaron los flujos funcionales de la aplicación web: análisis de texto pegado y archivos .eml, selección del modelo por idioma, importación mediante Gmail y navegación entre las vistas de detección y entrenamiento. Los resultados se presentan en la sección de pruebas y discusión.

Aspectos éticos y de protección de datos

El tratamiento de los correos electrónicos analizados por el prototipo se plantea conforme al Reglamento General de Protección de Datos (RGPD, Reglamento (UE) 2016/679). El sistema minimiza la información procesada: solo extrae los metadatos y las señales necesarias para la clasificación (cabeceras, remitente, asunto, enlaces y estructura HTML), sin almacenar de forma persistente el contenido íntegro de los mensajes, las credenciales de acceso ni los datos personales de los remitentes.

El acceso a la API de Gmail se realiza mediante OAuth, delegando en Google la autenticación del usuario y evitando el manejo de contraseñas en la aplicación. El prototipo está diseñado para entornos controlados y académicos, con datos de demostración o con el consentimiento explícito del usuario. Los tokens y credenciales se almacenan localmente y quedan excluidos del repositorio. Antes de publicar la aplicación para varios usuarios sería necesario incorporar autenticación, gestión segura de secretos y una política explícita de conservación de datos.

Diseño e implementación de la aplicación web

Tras el análisis teórico de las técnicas de phishing y de los principales enfoques de detección existentes, se ha desarrollado un prototipo funcional orientado al análisis de correos electrónicos sospechosos. El objetivo del sistema no es sustituir a una solución empresarial completa de seguridad, sino demostrar cómo puede combinarse un enfoque heurístico explicable con un clasificador automático basado en aprendizaje supervisado.

El prototipo se centra en el análisis de mensajes de correo electrónico desde una aplicación web. La entrada puede proceder de texto pegado manualmente, archivos .eml o correos importados mediante la API de Gmail. A partir de la entrada, el sistema extrae cabeceras, remitente, asunto, cuerpo en texto plano, contenido HTML, enlaces, anclas y adjuntos, y genera una evaluación de riesgo. Dicha evaluación puede realizarse mediante reglas heurísticas, mediante un modelo neuronal entrenado o mediante una combinación ponderada de ambos enfoques.

Alcance del sistema

El alcance del prototipo se ha delimitado al análisis estático del correo electrónico y de las URLs contenidas en él. El sistema no navega activamente hacia las páginas enlazadas, no descarga contenido externo ni consulta servicios de reputación en tiempo real. Esta decisión reduce los riesgos durante las pruebas y mantiene el prototipo en un entorno controlado. La entrada puede proceder de texto pegado, archivos .eml o la API de Gmail, tanto para importación manual como para comprobaciones desde la vista de monitorización.

No obstante, la arquitectura queda preparada para ampliaciones futuras. Podrían añadirse consultas a listas negras públicas, reputación de dominios, validación de certificados TLS, análisis dinámico de páginas o integración con sistemas corporativos de alerta. El prototipo no se plantea, por tanto, como un producto cerrado, sino como una base funcional y extensible.

Arquitectura general

La solución adopta una arquitectura modular autocontenida dentro de una única aplicación Python. Streamlit proporciona la capa de presentación y organiza las vistas de inicio, configuración, detección, monitorización y entrenamiento. Estas vistas invocan directamente los servicios del paquete sistema_phishing en el mismo proceso, por lo que el análisis no depende de un servicio HTTP separado ni requiere enviar el contenido del correo a un servidor central.

A nivel interno, el paquete sistema_phishing agrupa los componentes del dominio. analysis_service.py coordina los modos heurístico, neuronal y combinado y selecciona el modelo según el idioma del correo. analizador_email.py normaliza texto y archivos .eml; correo.py define la representación común; signal_builder.py construye las señales; scorer.py calcula la puntuación; explanations.py genera explicaciones; y modelo_neural.py encapsula el pipeline TF-IDF + MLP y su persistencia. Las vistas Streamlit consumen estas funciones mediante llamadas locales.

Las reglas heurísticas se han dividido en módulos especializados: header_signals.py para cabeceras y autenticación, content_signals.py para contenido textual y adjuntos, html_signals.py para estructuras HTML sospechosas y url_utils.py para extracción y análisis de URLs y dominios. La detección del idioma está centralizada en idioma.py, de modo que la aplicación y la monitorización seleccionan el modelo neuronal con el mismo criterio. Esta

división mejora la mantenibilidad y facilita añadir nuevas reglas sin modificar el núcleo del analizador.

Flujo de análisis

El flujo general comienza con la entrada del correo. Si el usuario sube un archivo .eml, el parser extrae remitente, destinatario, asunto, cuerpo en texto plano, cuerpo HTML, cabeceras, enlaces, anclas y adjuntos. Si el usuario pega el contenido manualmente, se genera una representación simplificada. Cuando la entrada procede de Gmail, la propia aplicación obtiene los mensajes autorizados mediante OAuth y los adapta al mismo formato interno.

A continuación, la entrada se normaliza en una representación interna común (CorreoAnalizado), de modo que las reglas heurísticas no dependen del formato original del mensaje.

Una vez normalizado el correo, el componente SignalBuilder ejecuta las reglas de detección y genera un diccionario de señales booleanas. Cada señal representa un posible indicador de phishing, como la existencia de un Reply-To diferente al From, fallos SPF/DKIM/DMARC, enlaces con dominios sospechosos, formularios HTML, redirecciones mediante JavaScript o presencia de lenguaje urgente. Si el modo de análisis lo requiere, se detecta el idioma del texto y se carga el modelo neuronal correspondiente.

RiskScorer asigna una ponderación a cada señal activa y calcula una puntuación final entre 0 y 100. Si la puntuación supera el umbral configurado, el mensaje se clasifica como phishing probable. ExplanationBuilder transforma las señales en explicaciones comprensibles. Todo el proceso se ejecuta localmente desde la vista Streamlit que inició el análisis. La monitorización de Gmail reutiliza el mismo servicio y, si se supera el umbral, puede enviar una alerta por Telegram y registrar el identificador del mensaje para evitar duplicados.

Sistema heurístico

El sistema heurístico constituye la parte más explicable del prototipo. Está formado por un conjunto de reglas diseñadas a partir de patrones habituales en correos de phishing. Estas reglas se agrupan en varias categorías.

En primer lugar, se analizan las cabeceras del correo. El sistema comprueba incoherencias entre From, Reply-To, Return-Path y Received-SPF, así como fallos de autenticación SPF, DKIM o DMARC. También se evalúan anomalías en cabeceras Received, dominios sospechosos en Message-ID y posibles intentos de suplantación mediante el nombre visible del remitente.

En segundo lugar, se analizan los enlaces. El sistema extrae URLs del texto y del HTML, comprueba la presencia de dominios en listas negras locales, detecta acortadores conocidos, identifica dominios con punycode o caracteres Unicode, y revisa parámetros de redirección sospechosos.

En tercer lugar, se estudia el contenido HTML. Se detectan formularios con acciones vacías, relativas o dirigidas a dominios sospechosos, así como elementos como iframe, base href, enlaces javascript: o redirecciones mediante meta refresh.

Por último, se analizan rasgos de ingeniería social presentes en el texto, como saludos genéricos, lenguaje urgente, asuntos sospechosos, solicitudes de credenciales o referencias a documentos adjuntos.

Clasificador neuronal

Además del sistema heurístico, se ha implementado un clasificador neuronal basado en scikit-learn. El modelo utiliza un pipeline formado por un vectorizador TF-IDF y un clasificador MLPClassifier. El vectorizador transforma el texto del correo en una representación numérica basada en n-gramas (por defecto unigramas y bigramas) con un máximo de características configurable. Los hiperparámetros de la red (capas ocultas, función de activación, regularización, learning rate, early stopping, etc.) están centralizados en la clase HiperparametrosModelo y pueden ajustarse desde el entorno (.env.local) o desde la interfaz de entrenamiento.

El sistema permite entrenar modelos en español e inglés, utilizando stopwords específicas para cada idioma. La aplicación de entrenamiento acepta uno o varios archivos CSV, permite configurar las columnas de texto, asunto, cuerpo y etiqueta, y admite formatos habituales de datasets públicos. Durante la evaluación se muestran la matriz de confusión y las métricas de precisión, recall y F1, además de la exactitud. También es posible entrenar hasta tres redes en memoria y compararlas sobre un conjunto de prueba, así como cargar varios CSV para combinar datasets.

Los modelos entrenados se almacenan en disco mediante joblib, diferenciando entre el modelo en español (modelo_neural_es.joblib) y el modelo en inglés (modelo_neural_en.joblib). Durante el análisis, el sistema detecta el idioma del correo con la librería langdetect y carga el modelo correspondiente a cada mensaje, no uno fijado para toda la sesión: un lote del monitor que mezcle correos en español e inglés utiliza el modelo adecuado para cada uno. Si no existe el modelo del idioma detectado, se reutiliza el del otro idioma y, en última instancia, un modelo sintético mínimo que mantiene la funcionalidad demostrativa.

Interfaz de usuario

La aplicación principal se desarrolla con Streamlit (src/app.py) y organiza la interfaz en cinco vistas: Inicio, Configuración, Detección, Monitor y Entrenamiento. La vista de detección permite analizar texto pegado, archivos .eml o correos importados desde Gmail y seleccionar el modo heurístico, neuronal o combinado. La vista de configuración gestiona Gmail, Telegram, los parámetros del monitor y los hiperparámetros del modelo. La vista de entrenamiento permite cargar CSV, entrenar hasta tres configuraciones y comparar sus métricas.

El resultado se muestra mediante una puntuación de riesgo, una clasificación global y un panel de detalles. En el análisis heurístico, el usuario puede consultar qué señales se han activado y leer una explicación de cada una. Esta decisión busca mejorar la transparencia del sistema, evitando que el resultado sea una simple etiqueta opaca.

La integración con Gmail se realiza desde la propia aplicación mediante OAuth. La vista de detección permite consultar e importar mensajes, mientras que la vista Monitor ejecuta comprobaciones bajo demanda y muestra el resultado de cada correo. De forma opcional, el proceso monitor_gmail.py reutiliza el mismo servicio local para realizar comprobaciones periódicas y enviar alertas por Telegram, sin introducir una capa de análisis remota.

Diseño modular

Durante el desarrollo se ha realizado una refactorización orientada a mejorar la mantenibilidad y aplicar principios de diseño SOLID. El sistema separa la presentación Streamlit, la coordinación de los casos de uso, la construcción de señales, la generación de

explicaciones, el cálculo de riesgo, el procesamiento de datasets y la lógica del modelo neuronal. La interfaz depende de servicios locales estables y no de detalles internos de cada regla o clasificador.

Esta organización permite que cada módulo tenga una responsabilidad principal. Por ejemplo, SignalBuilder no calcula puntuaciones, sino que únicamente construye señales; RiskScorer no conoce cómo se han obtenido las señales, solo calcula el riesgo; y ExplanationBuilder no decide si un correo es phishing, sino que transforma resultados técnicos en texto comprensible.

Esta separación facilita la evolución del prototipo. Pueden añadirse reglas en los módulos de señales, sustituirse el clasificador neuronal o incorporarse nuevas fuentes de entrada sin alterar la navegación de la aplicación. La revisión actual de la rama web ejecuta correctamente 33 pruebas automatizadas que protegen los componentes principales y sus integraciones locales.

Limitaciones

El prototipo presenta varias limitaciones. Las listas negras son locales y no se consultan servicios externos de reputación en tiempo real. Tampoco se realiza análisis dinámico de las páginas enlazadas, se validan certificados digitales ni se comprueba la reputación histórica de los dominios. Además, Streamlit se utiliza como interfaz académica en un entorno controlado: la versión actual no incorpora gestión multiusuario, control de acceso ni aislamiento de datos para un despliegue público.

El modelo neuronal depende de la calidad y representatividad de los datasets utilizados para el entrenamiento. Aunque la aplicación permite entrenar con CSV externos, su rendimiento real debe evaluarse con conjuntos amplios, recientes y equilibrados. Por último, el prototipo no incorpora transformers ni LLMs, aunque el estado del arte los identifica como una línea prometedora junto con la detección semántica de principios de ingeniería social.

Pruebas, resultados y discusión

Pruebas unitarias y de integración

La revisión actual de la rama web ejecuta 33 pruebas unitarias y de integración mediante unittest. Cubren el parseo de .eml, la construcción de resultados combinados, la carga de configuración, la decodificación de mensajes de Gmail, la monitorización, las reglas heurísticas, el clasificador neuronal, la persistencia de modelos y las notificaciones por Telegram. Todas finalizaron correctamente; los únicos avisos observados corresponden a iteraciones reducidas del MLP en pruebas rápidas y no implican fallos funcionales.

Dataset de ejemplo

Para probar el flujo completo de entrenamiento sin depender de información personal se preparó un conjunto sintético de 25 correos: 13 etiquetados como phishing y 12 como legítimos, en español e inglés. El conjunto incluye urgencia, solicitud de credenciales, enlaces sospechosos y mensajes legítimos de confirmación, reuniones o boletines. No pretende ser un corpus de producción, sino una muestra controlada para demostrar el entrenamiento y la evaluación.

Resultados del clasificador neuronal

Para obtener una estimación realista del comportamiento del clasificador neuronal sobre el dataset de ejemplo, se separó el conjunto en entrenamiento (70 %) y prueba (30 %), manteniendo la proporción de clases. Sobre la muestra de prueba (8 correos), el modelo alcanzó una exactitud del 75 %, una precisión del 67 %, un recall del 100 % y un F1 de 0,80: detectó todo el phishing del conjunto de prueba, aunque con algún falso positivo. Estas cifras deben interpretarse con cautela: el volumen de datos es muy reducido y no son representativas de un entorno real. Con el conjunto completo (evaluación sobre los mismos datos usados para entrenar), el modelo memoriza los ejemplos y alcanza valores cercanos al 100 %, síntoma clásico de sobreajuste que confirma la necesidad de datasets amplios y representativos.

Comportamiento del sistema heurístico

El sistema heurístico es deliberadamente conservador: solo clasifica como phishing probable los mensajes que superan el umbral (por defecto 45 sobre 100) acumulando varias señales. Sobre el dataset de ejemplo, construido con mensajes sintéticos sin cabeceras reales (sin Received ni resultados SPF/DKIM/DMARC), la heurística por sí sola detecta las señales de contenido (lenguaje urgente, saludo genérico, solicitud de credenciales o enlaces sospechosos) pero rara vez alcanza el umbral. Este comportamiento es el esperado y evidencia que sus mejores resultados se obtienen con correos reales que incluyen cabeceras de autenticación y enlaces a dominios conocidos: la heurística aporta explicabilidad, mientras que la generalización la aporta el modelo neuronal, de ahí el interés del modo combinado.

Prueba funcional de la aplicación web

Se realizó una prueba funcional del análisis invocado directamente por la aplicación web. Un mensaje sospechoso en español obtuvo una puntuación combinada de 50,2 sobre 100 y fue clasificado como phishing probable; un caso equivalente en inglés obtuvo 46,6 y activó correctamente el modelo del idioma detectado. Como control, dos mensajes legítimos equivalentes en español e inglés obtuvieron una puntuación de 0 y no fueron marcados. La

prueba confirma que la selección de idioma y los tres modos de análisis funcionan sin depender de un servicio remoto.

Discusión

Los resultados confirman el diseño como una base funcional: la heurística aporta señales comprensibles sobre contenido y cabeceras, el clasificador neuronal reconoce patrones aprendidos y el modo combinado equilibra ambas perspectivas. La principal limitación es el tamaño del conjunto de evaluación: un corpus reducido provoca sobreajuste y métricas poco generalizables. Para una evaluación rigurosa será necesario entrenar con datasets públicos amplios y recientes y comparar con métodos del estado del arte. Aun así, el prototipo demuestra la viabilidad de integrar reglas explicables y aprendizaje supervisado en una aplicación web autocontenida.

Conclusiones

El Trabajo Fin de Grado ha alcanzado los objetivos planteados. Se ha estudiado el phishing desde su base psicológica y su evolución reciente, se ha revisado el estado del arte en machine learning y deep learning y se ha desarrollado una aplicación web funcional que combina análisis heurístico explicable con un clasificador neuronal TF-IDF + MLP. La aplicación permite analizar texto pegado, archivos .eml y correos importados mediante la API de Gmail, así como entrenar y evaluar modelos desde una interfaz Streamlit integrada.

Para comprobar el cumplimiento de los objetivos, se revisa a continuación el grado de consecución de cada uno de los objetivos específicos planteados en la introducción:

El primer objetivo, analizar el fenómeno del phishing, sus fases, tipologías y evolución histórica, se aborda en el marco teórico mediante modelos de fases, vectores de ataque, modalidades según personalización y técnicas emergentes contrastadas con fuentes académicas e informes de industria.

El segundo objetivo, estudiar los fundamentos psicológicos de la ingeniería social, se cubre mediante el análisis de los principios de Cialdini, los sesgos cognitivos de Kahneman y Tversky y el impacto de la inteligencia artificial generativa.

El tercer objetivo, revisar el estado del arte en técnicas de detección, se desarrolla en la sección dedicada a machine learning clásico, deep learning, transformers y enfoques híbridos, y concluye con el hueco de investigación que justifica el prototipo.

El cuarto objetivo, diseñar e implementar un sistema heurístico explicable y un clasificador TF-IDF + MLP, se materializa en el motor de señales sobre cabeceras, contenido, URLs y HTML y en el pipeline neuronal entrenable en español e inglés.

El quinto objetivo, integrar la aplicación con Gmail y proporcionar alertas mediante Telegram, se cumple mediante el acceso OAuth incorporado en las vistas Streamlit y la monitorización opcional que reutiliza el mismo servicio local de análisis.

El sexto objetivo, evaluar el comportamiento del sistema mediante pruebas unitarias, de integración y funcionales, se aborda con la suite de 33 casos, la evaluación del clasificador con separación de entrenamiento y prueba y la comprobación directa de mensajes sospechosos y legítimos en español e inglés.

Entre los logros principales destacan la modularidad del código, la selección del modelo por idioma, la ejecución autocontenida del análisis, la integración de Gmail en la interfaz y la incorporación de métricas de exactitud, precisión, recall y F1 en la vista de entrenamiento.

El prototipo demuestra la viabilidad de un detector de phishing explicable y extensible dentro del alcance de un Trabajo Fin de Grado.

Trabajo futuro

Como trabajo futuro se plantea ampliar la evaluación con datasets públicos recientes en español e inglés y estudiar la robustez frente a ataques adversariales. En el plano del modelo, podrá sustituirse TF-IDF + MLP por transformers o LLMs con detección explícita de autoridad, urgencia y escasez y con explicaciones en lenguaje natural. También se contempla integrar reputación de dominios, análisis dinámico de páginas y etiquetado seguro de mensajes en Gmail. Para una publicación multiusuario de la aplicación web será necesario añadir autenticación, gestión de secretos, aislamiento de sesiones y una política de protección de datos.

Glosario

API: Conjunto de operaciones que permite integrar la aplicación con servicios externos, como Gmail o Telegram.

Aplicación web autocontenida: Aplicación que integra la interfaz y la lógica de análisis en un mismo entorno de ejecución.

Blacklist (lista negra): Lista local de dominios o URLs considerados no confiables que el sistema consulta durante el análisis.

Business Email Compromise (BEC): Fraude que suplanta a directivos o proveedores para autorizar pagos o transferencias fraudulentas.

DKIM: Mecanismo de autenticación de correo basado en firmas criptográficas del dominio remitente.

DMARC: Política que indica a los servidores receptores cómo tratar los mensajes que no superan SPF o DKIM.

Análisis combinado: Modo que pondera la puntuación heurística y la probabilidad del clasificador neuronal.

Streamlit: Framework de Python utilizado para construir la interfaz web interactiva del prototipo.

Heurística: Conjunto de reglas basadas en conocimiento experto que evalúan indicios de phishing de forma explicable.

MLPClassifier: Perceptrón multicapa de scikit-learn: red neuronal de alimentación directa con una o más capas ocultas.

OAuth: Protocolo de autorización empleado para acceder a la API de Gmail sin compartir la contraseña.

Phishing: Técnica de ingeniería social que suplanta entidades legítimas para obtener información sensible.

Precisión: Proporción de casos clasificados como phishing que realmente lo son.

Quishing: Variante de phishing que utiliza códigos QR como vector de entrega.

Recall: Proporción de los casos de phishing reales que el modelo detecta correctamente.

Smishing: Variante de phishing mediante mensajes SMS.

TF-IDF: Técnica de vectorización de texto que pondera la frecuencia de cada término según su importancia.

Umbral (threshold): Valor de puntuación a partir del cual un correo se considera phishing probable.

Vishing: Variante de phishing mediante llamadas telefónicas.

Referencias

- AlEroud, A., & Karabatis, G. (2020). Bypassing detection of URL-based phishing attacks using generative adversarial deep neural networks. In *Proceedings of the Sixth International Workshop on Security and Privacy Analytics (IWSPA '20)* (pp. 53-60). ACM. <https://doi.org/10.1145/3375708.3380315>
- Alghenaim, M., Alkaws, G., & Barnhart, C. R. (2025). The state of the art in AI-based phishing detection: A systematic literature review. In M. A. Al-Sharafi, M. Al-Emran, M. A. Mahmoud, & I. Arpacı (Eds.), *Current and Future Trends on AI Applications* (Studies in Computational Intelligence, Vol. 1178, pp. 431–458). Springer. https://doi.org/10.1007/978-3-031-75091-5_23
- Alkhalil, Z., Hewage, C., Nawaf, L., & Khan, I. (2021). Phishing attacks: A recent comprehensive study and a new anatomy. *Frontiers in Computer Science*, 3, Article 563060. <https://doi.org/10.3389/fcomp.2021.563060>
- Altawijry, N., Al-Turaiki, I., Alotaibi, R., & Alakeel, F. (2024). Advancing phishing email detection: A comparative study of deep learning models. *Sensors*, 24(7), 2077. <https://doi.org/10.3390/s24072077>
- Anti-Phishing Working Group. (2025). Phishing Activity Trends Report – 2nd Quarter 2025. https://docs.apwg.org/reports/apwg_trends_report_q2_2025.pdf
- Cialdini, R. B. (1984). *Influence: The Psychology of Persuasion*. William Morrow.
- Federal Bureau of Investigation. (2024). 2023 Internet Crime Report. Internet Crime Complaint Center. https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf
- Fortinet. (2025). 19 types of phishing attacks with examples. <https://www.fortinet.com/resources/cyberglossary/types-of-phishing-attacks>
- GreatHorn. (2021). 3 phases of the phishing attack kill chain. <https://web.archive.org/web/20231201201455/https://www.greathorn.com/blog/what-are-the-3-phases-of-the-phishing-attack-kill-chain/>
- Hadnagy, C. (2018). *Social Engineering: The Science of Human Hacking* (2nd ed.). Wiley.
- Heiding, F., Schneier, B., Vishwanath, A., Bernstein, J., & Park, P. S. (2024). Devising and detecting phishing emails using large language models. *IEEE Access*, 12, 42131-42146. <https://doi.org/10.1109/ACCESS.2024.3375882>
- Hoxhunt. (2024). The Phishing Kill Chain: Three Steps and Five Effects. <https://hoxhunt.com/blog/three-steps-and-five-effects-of-the-phishing-attack-kill-chain>
- Hoxhunt. (2025). Phishing Trends Report 2025. <https://hoxhunt.com/guide/phishing-trends-report>
- Hutchinson, S., Zhang, Z., & Liu, Q. (2018). Detecting phishing websites with random forest. In *Machine Learning and Intelligent Communications* (pp. 470-479). Springer. https://doi.org/10.1007/978-3-030-00557-3_46
- IBM. (2025). Cost of a Data Breach Report 2025. <https://www.ibm.com/reports/data-breach>
- Imperva. (2023). What is phishing: Attack techniques & scam examples. <https://www.imperva.com/learn/application-security/phishing-attack-scam/>
- Instituto Nacional de Ciberseguridad. (2025). Balance de ciberseguridad 2024. https://www.incibe.es/sites/default/files/Comunicación_2025/Infografía_BalanceCiberseguridad_INCIBE_2024_web.pdf
- Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- Khalifa, O. Y. I., & Muhammad Zaly Shah, M. Z. (2025). MultiPhishNet: A multimodal approach of QR code phishing detection using multi-head attention and multilingual embeddings. *International Journal of Innovative Computing*, 15(1), 53-61. <https://web.archive.org/web/20260118145104/https://ijic.utm.my/index.php/ijic/article/view/512>
- Koide, T., Fukushi, N., Nakano, H., & Chiba, D. (2024). ChatSpamDetector: Leveraging large language models for effective phishing email detection. *arXiv preprint arXiv:2402.18093*. <https://arxiv.org/abs/2402.18093>
- Kroll. (2025). Q4 2024 Cyber Threat Landscape Report: CorruptQR Campaign. <https://www.kroll.com/en/reports/cyber/threat-intelligence-reports/q4-2024-threat-landscape-report-phishing>
- Kuikel, S., Piplai, A., & Aggarwal, P. (2025). Evaluating large language models for phishing detection, self-consistency, faithfulness, and explainability. *arXiv preprint arXiv:2506.13746*. <https://arxiv.org/abs/2506.13746>
- Le, H., Pham, Q., Sahoo, D., & Hoi, S. C. H. (2018). URLNet: Learning a URL representation with deep learning for malicious URL detection. *arXiv preprint arXiv:1802.03162*. <https://arxiv.org/abs/1802.03162>
- LevelBlue. (2024). Dissecting a multi-stage phishing attack. <https://levelblue.com/blogs/security-essentials/dissecting-a-multi-stage-phishing-attack>
- LevelBlue. (2025). 2025 LevelBlue Futures Report: Cyber Resilience and Business Impact. <https://levelblue.com/2025-futures-report>

- Microsoft. (2025). Microsoft Digital Defense Report 2025. <https://www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025/>
- MITRE. (2025). ATT&CK Framework: T1566 Phishing. <https://attack.mitre.org/techniques/T1566/>
- OpenAI. (2023). GPT-4 Technical Report. <https://arxiv.org/abs/2303.08774>
- Popescul, D., & Radu, L. D. (2025). AI in phishing detection: A bibliometric review. *Frontiers in Artificial Intelligence*, 8, 1496580. <https://doi.org/10.3389/frai.2025.1496580>
- Proofpoint. (2025). What is phishing? Meaning, attack types & more. <https://www.proofpoint.com/us/threat-reference/phishing>
- Proofpoint. (2025a). The Human Factor 2025: Vol. 1 Social Engineering. <https://www.proofpoint.com/us/resources/threat-reports/human-factor-social-engineering>
- Proofpoint. (2025b). The Human Factor 2025 Vol. 2: URL Phishing. <https://www.proofpoint.com/us/resources/threat-reports/human-factor-url-phishing>
- Rojas-Galeano, S. (2024). Zero-shot spam email classification using pre-trained large language models. *arXiv preprint arXiv:2405.15936*. <https://arxiv.org/abs/2405.15936>
- Sánchez-Paniagua, M., Fidalgo, E., Alegre, E., & Alaiz-Rodríguez, R. (2022). Phishing websites detection using a novel multipurpose dataset and web technologies features. *Expert Systems with Applications*, 207, 118010. <https://doi.org/10.1016/j.eswa.2022.118010>
- Thakur, K., Ali, M. L., Obaidat, M. A., & Kamruzzaman, A. (2023). A systematic review on deep-learning-based phishing email detection. *Electronics*, 12(21), 4545. <https://www.mdpi.com/2079-9292/12/21/4545>
- Trad, F., & Chehab, A. (2024). Prompt engineering or fine-tuning? A case study on phishing detection with large language models. *Machine Learning and Knowledge Extraction*, 6(1), 367-384. <https://doi.org/10.3390/make6010018>
- Tversky, A., & Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157), 1124-1131. <https://doi.org/10.1126/science.185.4157.1124>
- Uddin, M. A., Mahiuddin, M., & Sarker, I. H. (2024). An explainable transformer-based model for phishing email detection: A large language model approach. *arXiv preprint arXiv:2402.13871*. <https://arxiv.org/abs/2402.13871>
- Verizon. (2021). The history of phishing attacks. <https://www.verizon.com/business/resources/articles/s/the-history-of-phishing/>
- Verizon. (2025). 2025 Data Breach Investigations Report. <https://www.verizon.com/business/resources/reports/2025-dbir-executive-summary.pdf>
- Wang, Y., Zhu, W., Xu, H., Qin, Z., Ren, K., & Ma, W. (2023). A large-scale pretrained deep model for phishing URL detection. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 1-5). IEEE. <https://doi.org/10.1109/ICASSP49357.2023.10095719>
- Wikipedia. (2025). Phishing. <https://en.wikipedia.org/wiki/Phishing>
- Yerima, S. Y., & Alzaylaee, M. K. (2020). High accuracy phishing detection based on convolutional neural networks. In *Proceedings of the 3rd International Conference on Computer Applications & Information Security* (pp. 1-6). IEEE. <https://doi.org/10.1109/ICCAIS48893.2020.9096869>
- ZeroFox. (2025a). 2025 Key Forecasts Report. <https://www.zerofox.com/reports/2025-key-forecasts-report/>
- ZeroFox. (2025b). 2025 Cyber Threat Predictions and Recommendations. <https://www.zerofox.com/blog/2025-cyber-threat-predictions-and-recommendations-from-zerofox-intelligence/>

Anexos

Este capítulo reúne el material complementario del Trabajo Fin de Grado: la estructura completa del repositorio, los fragmentos de código fuente más representativos del sistema y una guía resumida de instalación y uso. El código completo del prototipo se encuentra disponible en el repositorio GitHub asociado a este trabajo.

Anexo A: Estructura del repositorio

La siguiente estructura recoge los componentes principales que forman parte del alcance de la aplicación web y de su motor local de análisis:

Estructura de directorios del repositorio

```
TFG/
├── src/
│   └── app.py
# Entrada principal de Streamlit
```

— config_app.py	# Configuración de Gmail, Telegram y modelos
— detect_app.py	# Detección desde texto, EML y Gmail
— monitor_app.py	# Vista de monitorización de Gmail
— monitor_gmail.py	# Monitorización periódica opcional
— train_app.py	# Entrenamiento y evaluación
— ui_components.py	# Componentes visuales compartidos
— sistema_phishing/	
— analizador_email.py	# Parseo y normalización de correos
— analysis_service.py	# Coordinación del análisis local
— analyzer.py	# Orquestador heurístico
— configuracion.py	# Configuración del dominio
— content_signals.py	# Señales de texto y adjuntos
— correo.py	# Representación interna común
— dataset.py	# Normalización de CSV
— env_loader.py	# Variables locales de entorno
— explanations.py	# Explicaciones del riesgo
— gmail_client.py	# Integración OAuth con Gmail
— gmail_monitor.py	# Lógica de monitorización
— header_signals.py	# Cabeceras y autenticación
— heuristics.py	# Fachada heurística
— html_signals.py	# Señales HTML
— idioma.py	# Detección de idioma
— modelo_neural.py	# Pipeline TF-IDF + MLP
— neural.py	# Fachada neuronal
— scorer.py	# Cálculo de puntuación
— signal_builder.py	# Construcción de señales
— telegram_notifier.py	# Alertas por Telegram
— url_utils.py	# Análisis de URLs
— tests/	# Suite unittest
— scripts/experimentar_hiperparametros.py	
— modelo_neural_es.joblib	
— modelo_neural_en.joblib	
— requirements.txt	
— credentials.example.json	
— .env.example	

Anexo B: Fragmentos de código fuente

A continuación se reproducen los fragmentos más representativos del sistema: la fachada del análisis heurístico, el servicio común que combina heurística y red neuronal, y el núcleo del pipeline neuronal TF-IDF + MLP.

B.1. Fachada del análisis heurístico (heuristics.py)

heuristics.py

```
"""Fachada de heurísticas para el sistema de detección de phishing."""

from .analyzer import PhishingAnalyzer
from .signals import extraer_urls
from .correo import CorreoAnalizado

def analizar_correo(correo):
    """Analiza un correo y devuelve un informe de señales y riesgo.

    Esta función es la entrada pública del módulo heurístico: acepta tanto un
    texto plano como el diccionario generado por el parser de `.eml`.
    """
    # Se normaliza la entrada a una estructura común. Así el analizador
    # no necesita saber si el origen fue texto pegado o un fichero MIME.
    correo_analizado = CorreoAnalizado.from_input(correo)
    return PhishingAnalyzer(correo_analizado).analyze()
```

B.2. Servicio común de análisis (analysis_service.py)

analysis_service.py (extracto)

```
"""Servicio común de análisis heurístico, neuronal y combinado."""

from .analizador_email import construir_texto_para_analisis
```

```

from .heurísticas import analizar_correo
from .idioma import detectar_idioma_correo
from .modelo_neural import ModelStorage, NeuralPhishingClassifier,
    NeuralPhishingDetector

MODULO_HEURISTICO = "heurístico"
MODULO_NEURAL = "neural"
MODULO_COMBINADO = "combinado"

def construir_resultado_combinado(resultado_heur, resultado_neural, config):
    """Combina heurística y red neuronal con pesos configurables."""
    combined_score = (
        resultado_heur["risk_score"] * config.heur_weight
        + resultado_neural["risk_score"] * config.neural_weight
    ) / (config.heur_weight + config.neural_weight)
    return {
        "is_phishing": combined_score >= config.threshold,
        "risk_score": round(combined_score, 1),
        "urls": resultado_heur.get("urls", []),
        "signals": resultado_heur.get("signals", {}),
        "explanation": resultado_heur.get("explanation", []),
    }

class EmailAnalysisService:
    """Ejecuta el análisis configurado y reutiliza los detectores."""

    def __init__(self, config):
        self.config = config
        # Un detector cacheado por idioma (es/en) para no recargarlo
        # por cada correo del mismo ciclo.
        self._detectores = {}

    def analyze(self, datos_email):
        """Analiza un correo ya normalizado según el modo configurado."""
        resultado_heur = analizar_correo(datos_email)
        if self.config.mode == MODULO_HEURISTICO:
            return resultado_heur
        resultado_neural = self._analyze_neural(datos_email)
        if self.config.mode == MODULO_NEURAL:
            return resultado_neural
        return construir_resultado_combinado(
            resultado_heur, resultado_neural, self.config)

    def _analyze_neural(self, datos_email):
        texto = construir_texto_para_analisis(datos_email)
        idioma = detectar_idioma_correo(texto)
        detector = self._detectores.get(idioma)
        if detector is None:
            detector = cargar_detector_neural(self.config, idioma)
            self._detectores[idioma] = detector
        return detector.analyze(texto,
                                datos_email.get("from", ""),
                                datos_email.get("subject", ""))

```

B.3. Núcleo del clasificador neuronal (modelo_neural.py)

modelo_neural.py (extracto)

```

class NeuralPhishingClassifier:
    """Pipeline TF-IDF + MLP para clasificar correos como phishing o legítimos."""

    def __init__(self, max_features=5000, hidden_layer_sizes=(64,),
                 max_iter=300, random_state=42):
        self.pipeline = Pipeline([
            ("tfidf", TfidfVectorizer(max_features=max_features,
                                     ngram_range=(1, 2),

```

```

        stop_words=stopwords)),
        ("clf", MLPClassifier(hidden_layer_sizes=hidden_layer_sizes,
                               max_iter=max_iter,
                               random_state=random_state)),
    ])

def fit(self, textos, etiquetas):
    """Entrena el pipeline con textos vectorizados y sus etiquetas."""
    self.pipeline.fit(textos, etiquetas)
    return self

def predict_proba(self, texto):
    """Devuelve la probabilidad de que el texto sea phishing."""
    proba = self.pipeline.predict_proba([texto])[0]
    clases = self.pipeline.classes_
    # Normaliza la etiqueta "phishing email"/"safe email" por si el
    # dataset las registró invertidas.
    if "phishing email" in clases or "phishing" in clases:
        idx = list(clases).index(clases[0])
    return proba, clases

class NeuralPhishingDetector:
    """Envuelve un clasificador y devuelve un informe con la puntuación."""

    def __init__(self, classifier, threshold=0.5):
        self.classifier = classifier
        self.threshold = threshold

    def analyze(self, texto, remitente="", asunto=""):
        proba, clases = self.classifier.predict_proba(texto)
        score = round(float(proba[1]) * 100, 1) if len(proba) > 1 else 0.0
        return {
            "is_phishing": score >= self.threshold * 100,
            "risk_score": score,
            "description": "Resultado del clasificador neuronal TF-IDF + MLP.",
        }

```

Anexo C: Guía resumida de instalación y uso

Este anexo resume los pasos necesarios para ejecutar la aplicación web en un entorno local. La guía ampliada y la solución de problemas se mantienen en el archivo README del repositorio.

Comandos de instalación y uso

```

# 1. Crear y activar un entorno virtual
python -m venv .venv
.venv\Scripts\Activate.ps1

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Ejecutar la aplicación web
streamlit run src/app.py

# 4. Ejecutar la suite de pruebas
$env:PYTHONPATH="src"; python -m unittest discover -s tests -p "test_*.py"

```

C.1. Uso del prototipo

La aplicación web ofrece tres fuentes de entrada: texto pegado, archivos .eml y correos importados mediante la API de Gmail. El usuario selecciona el origen y el modo de análisis desde la vista Detección.

Para cada correo, el sistema devuelve una puntuación global de riesgo y el desglose de señales: dominio del remitente y autenticación SPF/DKIM/DMARC, urgencia en el asunto,

URLs sospechosas (acortadores, dominios recién registrados o direcciones IP), adjuntos y técnicas de ingeniería social. Las señales activadas se muestran con su peso heurístico, lo que permite interpretar por qué se ha clasificado un mensaje como phishing.

El clasificador neuronal asigna una probabilidad de phishing y selecciona el modelo según el idioma detectado. La interfaz Streamlit presenta conjuntamente la puntuación del modelo y las señales explicables de la heurística.

Desde la vista Monitor se pueden comprobar correos nuevos de Gmail y, si Telegram está configurado, enviar una alerta con el remitente, el asunto y la puntuación de riesgo.

Un caso de uso habitual: al recibir un correo sospechoso, el usuario lo carga como .eml o lo pega en la aplicación; el sistema lo clasifica en menos de un segundo y muestra las señales detectadas, permitiendo decidir con criterio si es seguro o debe reportarse.

Anexo D: Lista de verificación de la validación

La siguiente lista resume las comprobaciones realizadas sobre el prototipo durante la fase de validación, tal y como se detalla en el capítulo de pruebas y resultados.

- ☐ Comprobación de la suite de 33 pruebas unitarias y de integración.
- ☐ Normalización correcta de etiquetas phishing y correo legítimo.
- ☐ Entrenamiento del modelo TF-IDF + MLP en español e inglés.
- ☐ Evaluación con separación de entrenamiento (70 %) y prueba (30 %).
- ☐ Cálculo de exactitud, precisión, recall y F1.
- ☐ Arranque y navegación de la aplicación Streamlit.
- ☐ Análisis desde texto pegado y archivo .eml.
- ☐ Importación autorizada de correos mediante la API de Gmail.
- ☐ Selección del modelo neuronal según el idioma del mensaje.
- ☐ Monitorización de Gmail y notificación opcional por Telegram.